

Step 3.5 Flash : 以 11B 活跃参数实现开放的前沿级智能

StepFun Team

[GitHub](#) [HuggingFace](#) [ModelBlog](#)

Abstract

我们介绍 **Step 3.5 Flash**，一种稀疏 Mixture-of-Experts (MoE) 模型，旨在弥合前沿级智能体能力与计算效率之间的鸿沟。我们聚焦于构建智能体时最关键的因素：推理要精准，执行要快速且可靠。为此，Step 3.5 Flash 以 **196B** 参数的基础模型保障高保真建模，同时仅激活 **11B** 参数以实现高效推理，并通过交错的 **3:1** 滑动窗口/全注意力与 **多 Token 预测 (MTP-3)** 优化，显著降低多轮智能体交互的时延与成本。面向前沿级智能，我们设计了可扩展的 RL 框架，将可验证信号与偏好反馈融合，同时在大规模离策略训练中保持稳定，从而在数学、代码与工具使用上实现持续自我提升。Step 3.5 Flash 在智能体、编码与数学任务上表现强劲：IMO-AnswerBench 85.4%，LiveCodeBench-v6 (2024.08–2025.05) 86.4%， τ^2 -Bench 88.2%，BrowseComp (含上下文管理) 69.0%，Terminal-Bench 2.0 为 51.0%，整体性能与 GPT-5.2 xHigh、Gemini 3.0 Pro 等前沿模型相当。通过重新定义效率边界，Step 3.5 Flash 为在真实工业环境中部署高阶智能体提供了高密度的基础。

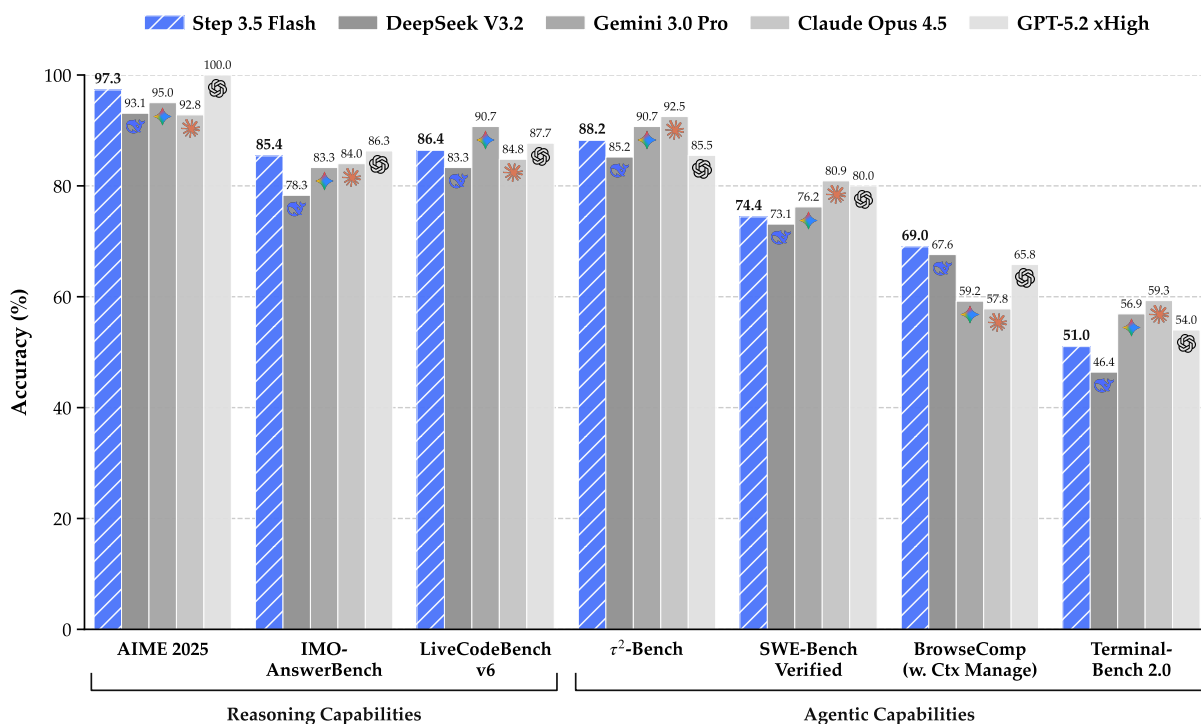


Figure 1: Step 3.5 Flash 仅以 11B 活跃参数 (196B MoE) 即可达到前沿级智能，性能可与领先的闭源与开源模型相当。

Contents

1 引言	4
2 架构	4

2.1	设计理念	4
2.2	带混合注意力的稀疏 MoE 骨干	5
2.3	架构消融与结果	7
3	基础设施	8
3.1	计算集群	8
3.2	训练框架	8
3.3	高吞吐轻量监控	9
4	预训练与中期训练	9
4.1	训练稳定性	9
4.1.1	Muon 的数值敏感性	10
4.1.2	超越路由崩溃的专家塌陷	10
4.1.3	MoE 层的局部激活爆炸	10
4.2	训练课程	11
4.2.1	数据混合	11
4.2.2	训练日程	12
4.2.3	超参数	12
5	后训练	13
5.1	专家模型构建与自蒸馏	13
5.2	可扩展 RL	13
5.2.1	MIS 过滤的策略优化 (MIS-PO)	13
5.2.2	奖励系统	14
5.2.3	超参数	15
5.3	数据合成与整理	16
5.3.1	通用与推理	16
5.3.2	通用化工具学习	16
5.3.3	代码智能体	16
5.3.4	搜索与研究智能体	17
5.4	智能体基础设施	17
6	评测	17
6.1	预训练评测	17

6.2 后训练评测	18
7 局限性	19
A Architecture Details	22
A.1 Head-wise Gated Attention	22
A.2 Speed Benchmark of Attention Enhancements	23
A.3 Meta Token	24
A.4 Pre-training Ablations Details	24
B Detail Analysis of Localized Activation Blow-up	25
C Step Pre-training Data Foundation	27
C.1 Knowledge Data Construction	27
C.2 Code Data	28
C.3 Mathematics & STEM Data	29
C.4 Data Infrastructure	29
C.5 Data Ablations Setting	30
D Post Training Details	30
D.1 SFT Details	30
D.2 RL Details and Ablations	30
D.3 Tool-integrated Reasoning and Parallel Reasoning	34
E Detailed Evaluation Protocols and Prompts	35
E.1 Evaluation Details of Pre-trained Models	35
E.2 Evaluation Details of Post-Trained Models	40
E.3 Internal Evaluation - Benchmarks and Methodology	47

1. 引言

尽管开源大语言模型 (LLM) [1–6] 已在可验证任务 [7–9] 上迅速缩小与闭源前沿系统 [10–12] 的性能差距，随着智能体系统的重要性上升，新的挑战随之出现。尤其是在复杂推理方面，开源模型仍落后于闭源前沿模型。此外，关键的效率瓶颈限制了其在长上下文智能体任务中的应用 [13–21]，更难在边缘或资源受限环境中部署。

在设计 Step 3.5 Flash 的架构时，我们聚焦于两个核心方面：效率与容量。我们采用稀疏 Mixture-of-Experts (MoE) [22–26] 架构，总参数量 196B，但每个 token 仅激活 11B；同时使用 3:1 的滑动窗口注意力 (SWA) [27] 与全注意力混合比例，以及多 Token 预测 (MTP-3) [3, 28–30] 以降低长上下文时延。为在混合注意力下以最小开销提升容量，我们将 SWA 层的 query 头数从 64 提升到 96，并引入按头门控注意力 [31]。该设计支持大规模线上部署，在 OpenRouter 首周于 Hopper GPU 上可稳定达到约 170 tokens/s 的速度¹。

在预训练方面，我们将稳定性视为第一优先级，并通过轻量级异步指标服务器与微批级持续日志构建了完整的可观测与诊断栈。该基础设施可系统识别并缓解大规模 MoE 训练的失效模式（例如，Muon 相关的精度敏感、专家塌陷 [32]、激活爆炸 [5, 33]）。结合改进的 Muon 优化器 [34]，我们在 17.2T 高质量且多样的 token 上实现稳定训练，仅出现一次短暂的损失峰值。在这一稳定训练机制下，Step 3.5 Flash Base 在数学、代码与知识基准上与更大规模模型（如 DeepSeek-V3.2-Exp Base [1] 与 Kimi-K2-Base [5]）保持竞争力。尤其在 SimpleQA [35] 上达到 31.6%，尽管参数仅为其三分之一，仍超过 DeepSeek-V3.2-Exp Base。

面向前沿级智能，当前的后训练系统面临两项紧密耦合的挑战：自蒸馏所需的领域专家迭代效率低 [1–4]，以及强化学习 (RL) 在 MoE 模型上的长程推理可扩展性受限。训练单一通用模型以覆盖多领域，往往牺牲领域专长；而维护多个专家模型又会造成碎片化与不可持续的多模型迭代成本。同时，随着推理轨迹加深，离策略采样中的细微 token 概率差异会累积为高方差梯度，这在 MoE 模型中尤为显著，因为专家路由引入更大的分布漂移并在前沿性能区间内破坏优化稳定性 [1, 36–38]。

为解决上述问题，我们提出基于共享 SFT 基座的统一大规模 RL 后训练方案。该框架在领域专精与全局综合之间交替，既能高效迭代专家，又保持单一高性能通用模型。专门的中期训练阶段将上下文窗口扩展至 128k，并通过合成数据强化核心智能体与推理能力，为下游后训练提供更优初始化。为在该统一框架内实现稳定可扩展的 RL，我们提出 Metropolis Independence Sampling-Filtered Policy Optimization (MIS-PO) [39, 40]，在 token 与轨迹层面采用离散分布过滤替代连续重要性加权。通过将优化限制在稳定的信任区域内，MIS-PO 显著降低梯度方差并保留有效学习信号，使 RL 能可靠扩展到长程推理与智能体行为。

尽管只有 11B 活跃参数，Step 3.5 Flash 仍在广泛的推理与智能体基准上与前沿模型和系统具有竞争力。标准推理设置下，在 IMO-AnswerBench [41] 上达 85.4%，在 LiveCodeBench-v6 (2024.08–2025.05) [9] 上达 86.4%；同时在长程、工具增强能力上表现稳健： τ^2 -Bench [15] 为 88.2%，BrowseComp（含上下文管理）[17] 为 69.0%，Terminal-Bench 2.0 [16] 为 51.0%。在 PaCoRe [42] 深度思考推理下，Step 3.5 Flash 在需要更长推理与多轮综合的基准上进一步提升。综合来看，Step 3.5 Flash 在推理与智能体场景中显著缩小了开源模型与前沿专有系统之间的差距。

2. 架构

2.1. 设计理念

Step 3.5 Flash 的架构体现了模型—系统协同设计的范式转变。除了传统的智能与成本目标，自治智能体时代还引入了第三个关键约束：推理时延。在交互式智能体工作流中 [43, 44]，更低时延直接缩短任务完成的墙钟时间；反之，在固定时间预算内也可通过测试时扩展 [42, 45–47] 提升智能。

¹<https://openrouter.ai>

智能体工作负载通常呈现独特特征：大量上下文预填充，随后是长时间、多轮交互式解码。因此，我们围绕三条相互耦合的轴向协同设计 Step 3.5 Flash 以降低墙钟时延：注意力（加速长上下文处理并与 MTP 具良好匹配）、稀疏 MoE（避免分布式部署中的拖尾导致吞吐下降）、以及多 Token 预测（MTP，用于通过投机解码实现快速生成）。

注意力。为加速预填充，我们采用混合注意力机制 [33, 48, 49]，以缓解长上下文处理的二次复杂度。在解码端，我们优先考虑与投机解码 [50] 的架构兼容性，因为验证效率是在带宽受限硬件上最关键的杠杆。基于这些考虑，我们做出两项注意力设计决策：

- 滑动窗口注意力（**SWA**）。我们选择 SWA [27] 而非线性注意力 [7, 51]，以最大化解码效率。尽管两者都具有线性复杂度，但线性注意力的状态更新机制会使投机解码所需的高效草稿树生成与并行树验证更复杂 [52–54]。相较之下，SWA 保持标准注意力语义，并天然适合通过 KV 掩码进行并行验证。此外，在缺乏证据表明线性注意力在智能体任务的长上下文建模上更优的情况下，我们认为窗口大小 $w=512$ 的 SWA 在内核效率与局部依赖捕获之间取得了良好平衡。
- 硬件对齐的分组查询注意力（**GQA-8**）。面向标准 8-GPU 服务器节点部署，我们将模型配置为 8 个 KV 头（GQA-8）[55]。这使 KV-cache 分片与 8 路张量并行对齐，并改善内存访问模式。关键在于，GQA-8 虽使注意力更受内存带宽约束，但也产生可吸收投机草稿与验证开销的计算余量，使多 Token 投机更激进而不会带来成比例的时延惩罚。

稀疏 MoE。在前馈侧，我们采用细粒度 MoE [22–26]，在保持容量的同时降低平均 FFN 计算。我们使用专家并行（EP）[25] 以实现可扩展部署。然而在 EP 下，端到端时延可能被路由不均导致的拖尾所主导：token 分配偏斜会把负载集中到少数专家及其所在 GPU，并在同步点上限制吞吐。为此我们提出 EP 组均衡 MoE 路由策略。

多 Token 预测（MTP）。为进一步降低自回归时延，我们引入多 Token 预测（MTP）[29, 56]，作为投机解码 [50] 的互补杠杆。为使投机更轻量，我们利用 SWA 与稠密 FFN [3] 简化 MTP 头。

我们还将模型规模约束在 200B 参数以内，使其在高端工作站 128GB 内存预算内可进行高性能推理。

2.2. 带混合注意力的稀疏 MoE 骨干

如图 2 所示，Step 3.5 Flash 采用 45 层稀疏 MoE Transformer 骨干（3 层稠密层 + 42 层 MoE 层），并配以专门的混合注意力层布局。每个 MoE 层包含 288 个路由专家与 1 个共享专家，top- k 路由器每个 token 激活 $k=8$ 个专家。该配置在保持大容量知识（总参数 196B）的同时，将单 token 激活限制为 11B，确保推理时延足够低以支持高响应的智能体交互。表 6 总结了 Step 3.5 Flash 的关键架构超参数。

混合注意力层布局。为在长上下文效率与长程连接性之间取得平衡，Step 3.5 Flash 采用受 [33, 49, 58] 启发的 3:1 交错注意力布局（SWA : Full），记作 S3F1。该配置重复四层结构：三层 SWA ($w=512$) 后接一层全 GQA-8。然而在初始实验中，朴素的交错策略在多个基准上始终劣于稠密注意力基线（表 10）。为在不增加实际开销的情况下弥补差距，我们采用两项互补增强：(i) 提升 SWA 的 query 头数量；(ii) 采用按头门控注意力 [31]。

SWA 的增强 Query 头。将 query 头数量从 64 提升到 96，可有效缓解从统一全注意力架构转为 S3F1 布局时常见的性能下降（表 10）。我们认为这几乎是“免费午餐”，因为在长文本场景中，朴素 SWA 的开销极小，即便我们显著增大了头数。

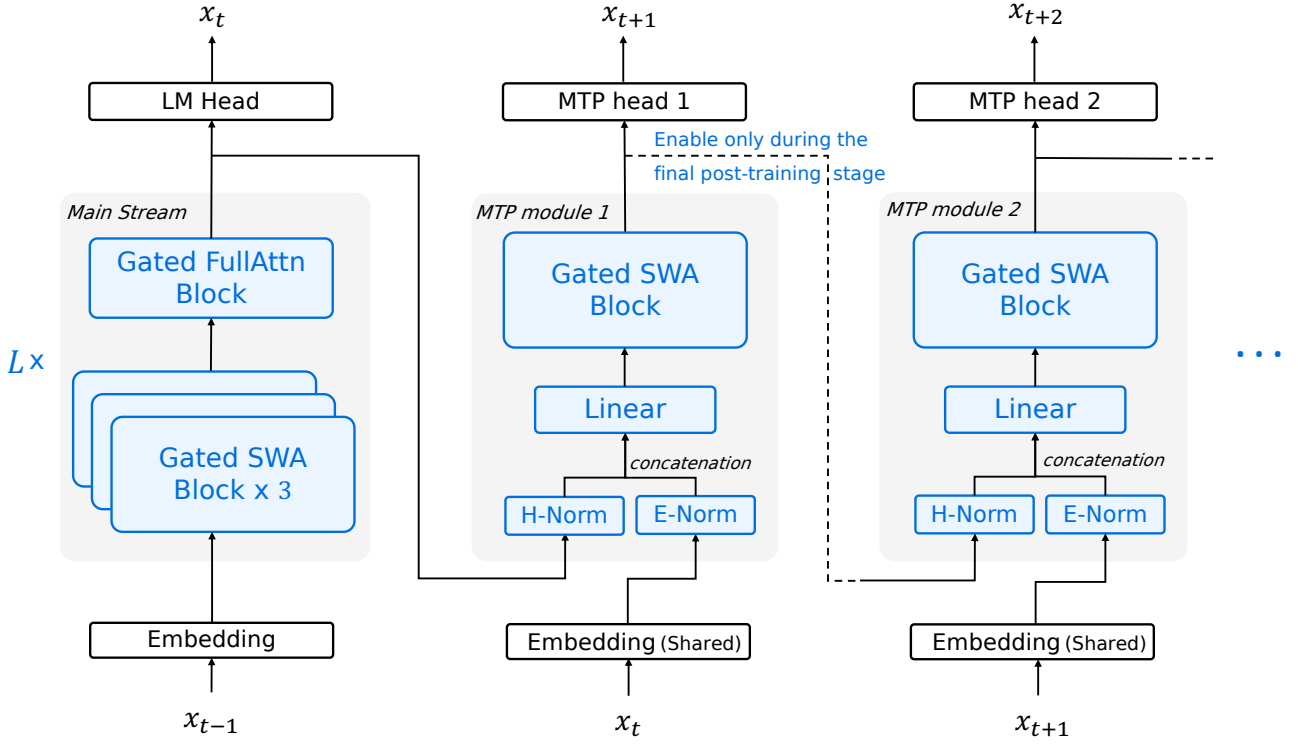


Figure 2: Step 3.5 Flash 架构示意图。模型采用按头门控注意力 [31]，以一个全注意力层起始，随后为 $L = 11$ 个混合块，每个混合块交错包含 3 个滑动窗口注意力（SWA）层与 1 个全注意力层（为清晰起见，图中省略第一层）。我们全程使用零中心 RMSNorm [57]。前三个块使用稠密 FFN，后续块采用稀疏 MoE FFN。MTP 模块使用 SWA 与稠密 FFN。为限制开销，主训练仅训练 MTP 模块 1；MTP 模块 2-3 由其拷贝，并在轻量级末期阶段联合微调。

按头门控注意力。朴素 SWA 的一个局限是：当输入窗口内缺乏有效信息时，它无法有效吸收多余的注意力权重 [31, 59–61]。已有工作 [3, 33] 通过在窗口内引入可学习、与数据无关的 sink token 来解决该问题。我们采用另一种方法：集成参数高效的按头门控机制 [31, 62, 63]，可视为引入数据相关的 sink token。实现细节与进一步讨论见附录 A.1。按头门控在理论 FLOPs 与实际时延上均可忽略。我们在附录 A.2 中给出了更多关于门控与增加 SWA 头数的性能分析与基准。

MoE 专家并行负载均衡。 我们采用无损负载均衡 [29, 64] 以鼓励专家之间的全局 token 平衡。然而该方法无法保证微批级别的 EP rank 负载均衡，可能导致拖尾并降低吞吐。因此我们引入 EP 级别的均衡损失，显式促进 rank 级别的均匀利用 [26]。

EP partitions experts $\mathcal{E}$ into G disjoint groups $\{\mathcal{E}_g\}_{g=1}^G$ across ranks. For token t , let S_t denote the top- K experts (mask $s_{t,e} = \mathbf{1}[e \in S_t]$) and $p_{t,e}$ the routing probabilities. Then, the EP load balancing loss $\mathcal{L}_{EP}$ is:

$$p_e = \frac{1}{T} \sum_{t=1}^T p_{t,e}, \quad f_e = \frac{1}{TK} \sum_{t=1}^T s_{t,e}, \quad p_g = \sum_{e \in \mathcal{E}_g} p_e, \quad f_g = \sum_{e \in \mathcal{E}_g} f_e, \quad \mathcal{L}_{EP} = G \sum_{g=1}^G f_g p_g. \quad (1)$$

多 Token 预测（MTP）。 为加速长上下文智能体工作负载下的投机解码，我们附加三个轻量级 MTP 头。每个 MTP 头由一个 SWA 和一个稠密 FFN 组成，仅增加 0.81B 参数（约 0.41%）。我们按相对标准 LM 头的额外预测偏移为这些头编号：对 $h \in \{1, 2, 3\}$ ，MTP- h 在位置 t 的骨干隐藏状态

布局	SWA 头数	相对 FLOPs 解码 / 预填充	预训练 平均	下游性能						
				推理	数学	代码	科学	通用	长上下文	平均
<i>FFFF</i>	32	~2.68 / 2.90	54.1	40.8	40.9	19.6	42.7	26.5	28.8	33.2
<i>S1F1</i>	32	~1.58 / 1.65	54.6	42.1	42.3	19.3	44.5	26.8	29.6	34.1
<i>S3F1</i>	32	1.00 / 1.00	53.6	40.2	40.4	18.9	42.4	25.4	27.5	32.5
<i>S3F1+Head</i>	48	~1.01 / 1.02	55.7	40.6	40.3	18.3	44.0	26.0	28.2	32.9

Table 1: 30B-A3B 的下游结果。*F* 表示全注意力，*S* 表示 SWA。*S3F1* 指混合布局中 3 个 *S* 层后接 1 个 *F* 层。相对 FLOPs 以 *S3F1* 为基准，并在 64k/256k 上下文上取平均（表 8）。Pre-train Avg. 汇总了通用、数学与代码基准（表 16）。

条件下预测 token x_{t+1+h} 。为控制训练开销，大多数训练阶段仅激活并优化 MTP-1。当骨干训练充分后，我们用 MTP-1 初始化 MTP-2 与 MTP-3，并在轻量级后训练阶段联合训练所有 MTP 头。受 Fast-MTP [65] 启发，我们在 MTP 头中对不同预测偏移采用位置相关的损失重加权，以避免过度优化远距 token 预测。

2.3. 架构消融与结果

我们进行了大量实验以验证 Step 3.5 Flash 的关键设计选择，重点关注 (i) 注意力布局（包含 SWA 与头数扩展），以及 (ii) 按头门控注意力与 sink token 的对比。为确保效率优化不损害模型性能，我们采用两种互补的消融协议：一种评估覆盖预训练、32k 长上下文扩展与 64k 上下文长度 SFT 的端到端流程；另一种将规模扩展到 100B 参数，以研究这些设计在规模化下的表现。所有表格的详细架构与评测设置见附录 A.4。下文总结了大规模实验的关键发现。

SWA 与长上下文。 我们训练 30B-A3B 模型完成完整流程（1.4T token 预训练后接 SFT），以评估混合注意力对推理与长上下文性能的端到端影响。我们消融四种注意力布局：全注意力（*FFFF*）、SWA/全注意力交替（*S1F1*）、3:1 的 SWA: 全注意力布局（*S3F1*），以及增加 SWA query 头的 *S3F1* 变体（*S3F1+Head*）。为隔离注意力结构影响，我们固定 SWA 窗口为 $W=512$ 并关闭 MTP（见附录表 9 与表 10）。

表 1 显示了清晰的成本—质量权衡。*S3F1* 的注意力侧 FLOPs 最低（预填充与解码分别归一化为 1.00），而 *FFFF* 约为其 $2.68\times/2.90\times$ ；但 *S3F1* 质量存在一致下降（*e.g.*，LongCtx 从 28.8 降至 27.5）。

增加 SWA 的 query 头数基本弥补了该损失。尤其是 *S3F1+Head* 在预训练阶段已超过 *FFFF*（55.7 vs. 54.1），后训练后仍具竞争力：LongCtx 从 27.5 提升到 28.2，Sci 从 42.4 提升到 44.0，以几乎可忽略的额外注意力成本弥合了与 *FFFF* 的差距。剩余劣势较为局部（*e.g.*，Code 小幅下降到 18.3），总体质量趋势更有利于 *S3F1+Head*。

有趣的是，交替的 *S1F1* 布局在总体 SFT 质量与 LongCtx 得分（29.6）上最佳，但需要更高的注意力侧预填充/解码 FLOPs（约 1.58/1.65），相较 *S3F1+Head* 成本上升约 60%。因此我们选择 *S3F1+Head* 作为长上下文智能体工作负载的默认配置，在保持强而稳定的长上下文性能的同时优先降低预填充/解码成本。

按头门控注意力与 Sink Token。 我们在 100B-A10B MoE 上进行缩放且受控的预训练实验，以研究现实规模条件下的注意力侧机制。具体而言，我们在保持相同 *S3F1* 布局（窗口 $W=512$ ）的前提下，对比 *sink token* 与 按头门控注意力。如表 2 所示，按头门控一致提升质量，使平均表现从 62.46 提升到 64.43（+1.97）。因此我们在后续研究中将按头门控注意力作为默认机制。

方法	BBH	MMLU	GPQA	MBPP	C-EVAL	CMMLU	平均
Sink Token	70.6	65.1	27.2	61.2	76.2	74.6	62.5
Head-wise Gate	73.7	67.0	28.1	62.6	77.9	77.1	64.4

Table 2: S3F1 布局下 100B-A10B 模型的仅预训练评估。按头门控在各基准（含总体平均）上一致优于固定 sink token。

3. 基础设施

3.1. 计算集群

Step 3.5 Flash 在由 4,096 张 NVIDIA H800 GPU 组成的大规模集群上训练。每个节点包含 8 张 GPU，通过 NVLink 与 NVSwitch 互连以实现高带宽的节点内通信。节点间互联则依赖 8×200 Gbps RoCE 链路，以保证大规模同步与数据交换效率。

3.2. 训练框架

Step 3.5 Flash 的训练由内部框架 *Steptron* 驱动，这是一个构建在 PyTorch [66] 与 Megatron-LM [67] 之上的轻量高性能系统。Steptron 统一了完整的模型研发流程，在同一工程栈下支持大规模预训练、后训练与强化学习（RL）工作负载。

Step 3.5 Flash 采用混合并行化策略，包括 8 路流水线并行（PP）[68]（带虚拟流水线阶段 VPP）、8 路专家并行（EP）[25]，以及 ZeRO-1 数据并行（DP）[69]。为实现 Step 3.5 Flash 的高效训练，我们采用以下工程技术。

解耦并行。 遵循 Megatron-Core [70]，我们实现了解耦并行方案，使注意力与 MoE 模块可使用不同的并行策略。我们为其分配独立的并行组，并在各自对应的数据并行组内完成梯度归约与缩放。

通信优化。 在解耦注意力与 MoE 的设置下，并发的 DP 通信流可能使 RoCE 链路饱和，因拥塞显著增加 DP 开销。为此我们提出两项互补的通信优化，整体迭代时间最多可降低 5%。其一，面向网络的通信调度将 DP 流量划分为节点内 NVLink 与节点间 RoCE 两个阶段并进行流水化，以充分利用两种互联；其二，通信感知的 *rank* 放置利用作业级通信画像跨交换机放置 rank，减少跳数并避免跨交换机热点。

Muon ZeRO-1 重分片。 Muon [34] 的 Newton-Schulz 正交化需要每个参数的完整（未分片）梯度，这与 ZeRO-1 [69] 的 reduce-scatter（将参数梯度分片到各 DP rank）相冲突。Megatron-LM 当前实现通过对 FP32 梯度做全量 all-reduce 来重建完整梯度后再进行 Muon 更新，但这几乎使通信翻倍。我们改为将完整参数分配给 DP rank，并将梯度缓冲区重新打包为按 rank 排布的缓冲区，使单次 reduce-scatter 即可把每个参数的完整梯度传递给其所属 rank。由于填充到最大 rank 的开销随 DP 规模增长，我们仅对专家参数应用此策略，对非专家参数仍使用 DP all-reduce。与朴素 all-reduce 基线相比，该混合策略仅增加不到 4 GB 额外内存即可将端到端迭代时间降低约 5%。

GPU 内核优化。 我们还进行内核级优化以提升训练效率。在注意力中，我们融合 QK 归一化与 RoPE。在 MoE 中，我们融合多个小算子以减少内核启动开销与内存流量，并实现基于分组 GEMM 的融合式 MoE gather/scatter，类似于 SonicMoE [71]。

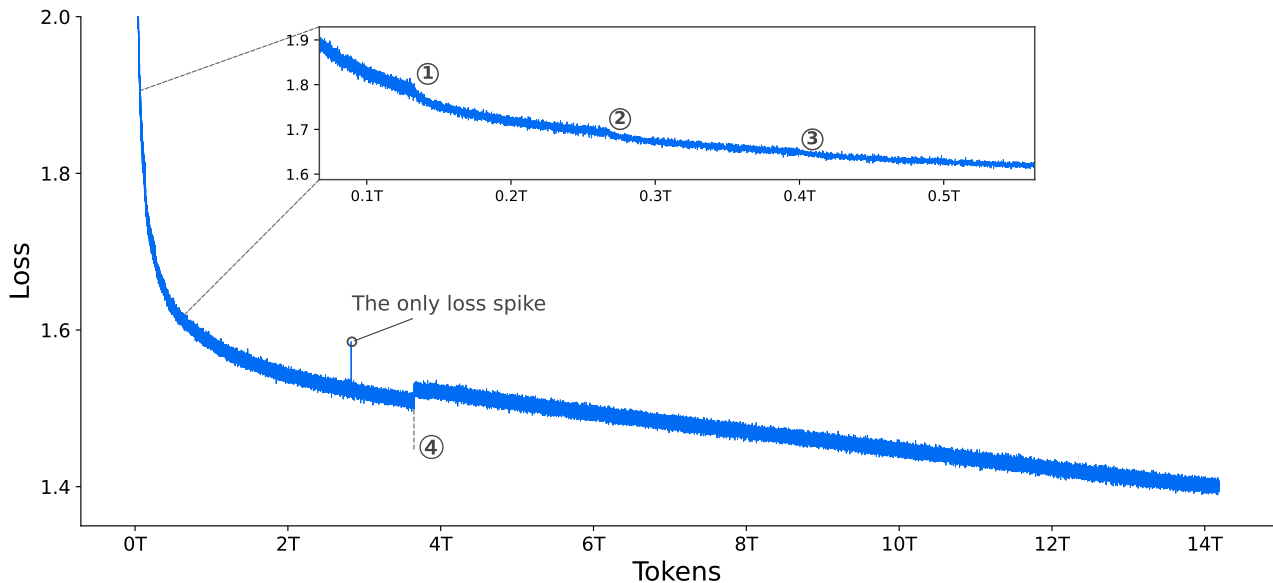


Figure 3: Step 3.5 Flash 的逐步训练损失曲线，未进行平滑或子采样。全程仅观察到一次孤立损失峰值。为清晰起见，省略了初始训练步。标记 ①–③ 分别表示 batch size 增至 8,192、12,288 与 16,384。标记 ④ 表示对 meta token 启用损失掩码（详见附录 A.3）。

细粒度选择性检查点。我们的训练框架支持细粒度激活重计算，提供按层、子模块级别的开关（*e.g.*，注意力、FFN、归一化、SiLU 与 MoE 置换），仅对最耗内存的组件进行选择性地重计算，以最小开销降低峰值内存。

3.3. 高吞吐轻量监控

我们收集全面的指标（例如每个微批内的专家分布、梯度范数）以进行细粒度训练监控。然而遥测规模巨大：4,096 张 GPU 的工作负载每次迭代产生近 600 万条消息。若在主循环内进行同步全局归约，将引入数秒开销，几乎使迭代时间翻倍，这对高性能训练不可接受。为此我们开发了轻量指标服务器，将遥测处理与训练路径解耦。每个 rank 使用自研异步通信框架 *StepRPC*，异步将本地指标卸载到远端服务器，使遥测开销降至约 100 ms/迭代。

指标服务器会缓存传入指标，只有在收到所有参与 rank 的迭代结束信号后才触发归约与入库，从而消除主循环中的同步。为在低延迟下摄取与处理数百万条消息，服务器采用高并发多进程系统，并包含两个解耦模块：(i) 面向高吞吐接入的消息接收器；(ii) 负责聚合与持久化的归约处理器。通过在模块内与模块间利用多核并行，服务器能够跟上遥测流并确保指标管理不滞后于训练。

4. 预训练与中期训练

概述。本节总结我们的预训练与中期训练流程，重点强调大规模稀疏 MoE 训练的实际稳定性约束。我们首先介绍训练稳定性诊断与缓解方法（第 4.1 节），随后给出预训练与中期训练的课程设计，包括数据混合、训练日程与关键超参数（第 4.2 节）。

4.1. 训练稳定性

训练稳定性是大规模稀疏 MoE 预训练的第一优先级。为使稳定性可操作，我们基于轻量级异步指标服务器与微批级连续日志构建了完整的可观测与诊断栈（见第 3.3 节）。该基础设施为优化器级与专家级信号提供细粒度可视性，从而系统缓解大规模 MoE 训练的重复性失效模式。

实践中，我们发现三类主导不稳定性，指标栈可帮助尽早发现并精确定位：(i) 由 Muon [34] 在低精度下对极分解迭代的数值敏感性引起的瞬时损失峰值与偶发随机数值爆炸；(ii) 即便路由分配统计看似健康也可能出现的专家侧塌陷（“死专家”）；(iii) 仅限于少量专家的局部激活爆炸。

在这些诊断指导下实施缓解措施后，预训练损失在全程保持平滑，仅出现一次损失峰值。图 3 展示了学习率退火前的完整曲线。

4.1.1. Muon 的数值敏感性

Muon 通过 Newton-Schulz (NS) 迭代 [72] 近似半正交更新方向。早期实验显示，采用收敛更快的正交化近似可带来稳定、适度的损失下降。因此我们使用 Polar Express [73] 迭代，并固定 $T=6$ 步，以在优化质量与吞吐之间平衡。

然而，即便采用推荐的安全缩放 [73]，我们仍偶尔观察到尖锐且不可恢复的损失峰值。该峰值具有非确定性（通常可通过从相邻 checkpoint 继续训练而避免），暗示潜在数值病理。仿真显示，bf16 的 Polar Express 在某些更新统计下会因加法累计误差极少数地产生极端中间异常值。因此我们仅将 Polar Express 迭代（状态与中间量）转换为 float16，其余训练保持混合精度。修改后该峰值不再出现。

4.1.2. 超越路由崩溃的专家塌陷

我们此前的 Step-3 [32] 报告 MoE 训练可能出现“死专家”，通常指专家长时间获得极少 token 分配，从而几乎没有有效梯度信号。在进一步研究中，我们发现即便路由分配保持稳定，专家塌陷仍可表现为专家侧病理，即专家激活消失、专家参数范数停滞或衰减。

我们观察到两项因素尤为关键：(i) 路由专家聚合需要显式缩放。当引入共享专家时，必须引入显式缩放因子来校准共享专家与路由专家的相对贡献。较小模型可能隐式学会此平衡，但大模型在自校准上不可靠。不匹配会抑制路由专家的有效贡献，即便路由频率看似健康。(ii) 细粒度稀疏下的微批均衡可能过于严格。对于细粒度稀疏 MoE 设计，微批级负载均衡约束（如 Switch 风格路由 [22]）可能过于苛刻。正如 [74] 所分析的，微批 LBL 可能引发过度的跨专家竞争并阻碍有效专精。

因此我们更倾向于更大范围的均衡（*e.g.*，全局批统计）[74,75] 或基于观测负载的无损偏置调整 [29,64]。实践中，路由分配统计通常稳定，并非专家塌陷的敏感指标。我们建议监控专家侧信号，包括每个专家的激活范数（*e.g.*，MoE FFN 中间层的 RMS/均值范数）与参数范数（*e.g.*，专家投影矩阵的 Frobenius 范数）。当部分专家的激活/更新趋近于零而中位数保持稳定（*e.g.*，最小值与中位数之比下降）时，可作为“专家死亡”的早期预警。

4.1.3. MoE 层的局部激活爆炸

随着主训练阶段专家专精逐渐成熟，我们在更深的 MoE 层观察到局部稳定性病理。具体而言，少量专家（常常每层仅一到两个）的激活范数快速增长，而同层大多数专家保持良好状态。这种差异导致激活分布呈重尾：专家激活范数的中位数稳定，但最大值爆炸，显著增加数值溢出和下游不稳定风险。

图 4 展示了该失效模式。值得注意的是，这种内部不稳定完全被训练损失掩盖，即便面板 (a) 中范数爆炸，损失几乎无变化。我们通过监控每个专家 FFN 输出范数的离散度来跟踪该现象。如面板 (b) 与 (c) 所示，中间层（*e.g.*，第 38 层）分布稳定，而最终层（*i.e.*，第 45 层）最大值（实线）与中位数（虚线）之间的差距迅速扩大。这表明激活能量危险地集中在深层网络中的少数“离群”专家上。为缓解该问题，我们评估了两种干预：

- 专家投影权重裁剪：约束 MoE FFN 专家投影矩阵的范数。对每个专家投影矩阵 W ，若其最大激活范数 $\max_x \|Wx\|$ 超过阈值 τ ，则按 $W \leftarrow W \cdot \frac{\tau}{\max_x \|Wx\|}$ 缩放。该方法类似注意力中的 MuonClip [5]，但我们在 checkpoint 上离线裁剪而非在线裁剪。

- 专家内部激活裁剪：在输出投影前对 MoE FFN 中间激活进行逐元素裁剪，方法同 [33]。

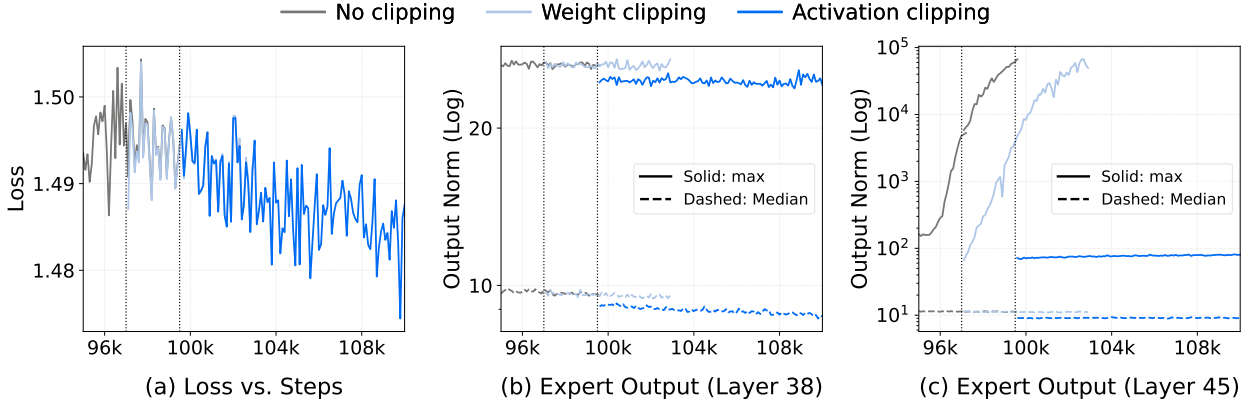


Figure 4: 专家激活稳定性与缓解策略分析。在面板 (b)–(c) 中，实线表示专家输出范数最大值，虚线表示中位数。(1) 深度相关不稳定：不同方法的训练损失几乎一致（面板 a），中间层保持稳定（*e.g.*，面板 b 的第 38 层），但最终层（*i.e.*，面板 c 的第 45 层）在无裁剪基线中出现灾难性范数爆炸。(2) 缓解：权重裁剪仅延缓爆炸，而激活裁剪能有效约束最大范数，确保各层稳定。

尽管图 4(a) 中不同缓解策略的训练损失几乎无差异，最大值与中位数之比却能可靠揭示潜在不稳定。如面板 (b) 与 (c) 所示，激活裁剪可确保内部范数稳定，而仅使用权重裁剪无法防止离群专家再次出现。因此，我们将每个专家激活范数的最大值/中位数比作为监控训练稳定性的关键且必要指标。

激活爆炸由多种因素驱动。我们观察到高频双字 (bi-gram) 可能触发专家专精。在使用 pre-norm [76,77] 时，单个专家可无限放大输出并主导最终输出范数，导致近确定性预测行为。SwiGLU [78] 加剧了这一风险：门控分支与上投影分支强对齐会产生稀疏且极端幅度的激活。Muon 通过放大持续的低秩更新进一步加速塌陷。详细分析见附录 B。

4.2. 训练课程

训练从广泛的开放域覆盖逐步走向更强的智能体与长上下文专精。我们先在 4k 上下文上进行广域开放域预训练以建立通用能力，然后在将上下文扩展到 32k 的同时，将数据混合退火到更高质量知识与更多软件开发数据（代码、PR、issue 与提交）。随后，中期训练阶段将上下文窗口从 32k 扩展至 128k，以强化长程推理并改善下游后训练与智能体工作负载的初始化。总体而言，我们在预训练中使用约 17.6T token，在中期训练中使用约 750B token。

4.2.1. 数据混合

我们的语料结合了通用开放域数据与面向智能体的数据。下文概述关键来源，更多细节见附录 C。

通用知识数据。 为支撑广泛的世界知识，我们构建了 **StepCrawl**（见附录 C.1.1），这是一个超越标准 Common Crawl [79] 的自研爬取与整理基础设施，可从网页 (HTML) 与书籍/文档类来源 (ePub/PDF) 规模化采集数万亿高质量 token。所有内容均经过多阶段质量过滤、站点/类别标注、去重与清洗。

代码数据。 强大的代码能力是智能体模型的基础。我们的代码语料使用改进的 OpenCoder [80] 流水线整理与精炼。我们将过滤策略从零容忍放宽至每篇文档允许 0–6 次启发式违规（附录 C.2.1），在质量与多样性之间平衡，并在退火与中期训练中上采样代码相关数据以增强智能体编程能力。

PR/Issue/Commit 数据。为更好匹配真实软件工程流程，我们从 GitHub 星标 10+ 的仓库中整理了全面的 PR/Issue/Commit 数据集（附录 C.2.2）。其中包括：(1) 经过 `git diff` 验证的基础数据（并与基准去重 [14, 81]）；(2) 基于 PR 讨论与提交、使用 Agentless 风格模板 [82] 生成的 **PR** 对话数据，用于文件定位与代码修复；以及 (3) 用于中期训练与后训练的衍生软件工程语料。

工具使用与推理数据。为提升工具使用鲁棒性与多步推理能力，我们加入覆盖数学/代码/科学/通识的合成与半合成数据，以及面向搜索智能体、SWE 智能体与工具执行的领域样本。在中期训练中，我们进一步引入长上下文样本（自然长文档与长形式合成任务），以强化长上下文规划与推理。

4.2.2. 训练日程

预训练日程。预训练包含两个阶段：

1. 预训练阶段 1：开放域预训练（14.6T token，4k 上下文）。进行广泛开放域训练，以最大化覆盖面与基础能力。
2. 预训练阶段 2：退火 + 长上下文初始化（3T token，4k 到 32k 上下文）。我们将数据混合退火到以代码与 PR/Issue/Commit 为中心的来源，同时提高高质量知识与高推理密度样本的比例。该阶段先在 4k 上下文训练 2T token，随后在相同退火混合下切换到 32k 上下文训练 1T token，以初始化长上下文训练。

中期训练日程。中期训练同样包含两个阶段：

1. 中期训练阶段 1：32k 专精（386B token，32k 上下文）。为缓解分布漂移并稳定专精，我们回放预训练中的 81B token（21%），同时强调软件工程师与工具使用相关混合。
2. 中期训练阶段 2：长上下文专精（364B token，128k 上下文）。我们保留 10.5B 回放 token，并进一步通过合成长程推理与自然长文档（从预训练数据中选择长度 > 32k 的样本），加上面向代码智能体、搜索智能体与工具使用的领域数据，强化长上下文能力。

4.2.3. 超参数

预训练超参数。预训练全程使用 Muon 优化器 [34]，权重衰减设为 0.1，梯度裁剪设为 1.0。学习率在前 2,000 步线性从 0 预热至 2.5×10^{-4} ，随后在预训练阶段 1 内采用余弦衰减至 5×10^{-5} 。在预训练阶段 2 中，我们在 4k 部分（2T token）对学习率从 5×10^{-5} 进行二次余弦衰减至 2×10^{-5} ，并在 32k 部分（1T token）将学习率固定为 2×10^{-5} 。全局 batch size 在前 400B token 内从 4096 逐步提升至 16384，并在后续训练保持 16384；在退火的 32k 部分设为 2k。MTP 损失权重在预训练阶段 1 设为 0.3，在预训练阶段 2 设为 0.1，遵循 [29]。对于无损负载均衡，偏置更新率在前 14.6T token 设为 0.001，并在退火期间衰减至 0.0；同时全程应用系数为 0.001 的 EP 组均衡损失。RoPE [83] 设置为：在 4k 训练期间全注意力与 SWA 均使用 $\theta = 10,000$ ；在退火的 32k 部分，仅对全注意力设置 $\theta_{\text{Full}} = 1,000,000$ ，并保持 $\theta_{\text{SWA}} = 10,000$ 。

中期训练超参数。中期训练继续使用 Muon [34]。我们冻结 MoE 路由器权重，禁用 EP 组均衡损失，并将 MTP 损失权重固定为 0.1（两阶段一致）。学习率在前 3% 迭代中从 0 预热至 2×10^{-5} ，在中期训练阶段 1 保持不变，在中期训练阶段 2 衰减至 7.3×10^{-6} 。在 RoPE 的选择性缩放中，中期训练阶段 1（32k）设 $\theta_{\text{Full}} = 1,000,000$ ，中期训练阶段 2（128k）提升至 $\theta_{\text{Full}} = 5,000,000$ ，而 $\theta_{\text{SWA}} = 10,000$ 在中期训练全程保持不变 [84]。

5. 后训练

本节介绍用于大规模强化学习 (RL) 的统一后训练方案，起点是统一的监督微调 (SFT) 模型。该框架将可验证奖励信号与人类偏好反馈结合，并在 MoE 模型的大规模策略训练中保持稳定，从而实现持续的自我提升。流程采用与先前工作 [2, 85] 类似的两阶段策略。首先，我们在统一 SFT 基线之上进行领域特定 RL，构建专家模型，覆盖数学、代码、STEM、工具使用、长上下文理解、人类偏好与智能体推理。随后通过自蒸馏与可扩展 RL 将这些专家蒸馏为通用模型，确保最终模型在多任务上与专门化基线保持竞争力。通过在目标化专精与全局综合之间系统交替，我们在不牺牲专家级性能的情况下获得稳健泛化。

5.1. 专家模型构建与自蒸馏

我们采用两阶段 SFT 流水线，为后续 RL 构建稳健基础。第一阶段进行大规模多领域 SFT，覆盖数学、代码、STEM、逻辑、通用问答、代码智能体、工具使用、搜索智能体与长上下文理解。通过难度感知过滤与策略性配比，促进广泛的智能体行为。第二阶段通过注入分布外 (OOD) 信号 [46, 86] 显式提升推理密度，包含约 30k 条专家级化学轨迹与合成算术任务。这种针对性的推理模式暴露可在仅 3 个 epoch 内激活潜在能力，为后续领域特定 RL 提供所需的结构复杂度初始化。

在完成领域特定 RL 后，我们将分化的专家能力汇聚为统一的学生模型，并从中期训练 checkpoint 初始化。在该阶段，专家模型使用与第一阶段 SFT 语料共享的提示分布生成高质量轨迹，相较直接并入 RL 更稳定、更高效。该方法通过拒绝采样剔除语言混杂或过度思考等不良模式，将专家知识集中到单一学生模型中。由此建立的高质量基础显著降低了后续 RL 的优化负担。

超参数。采用 Muon 优化器 [34]，3% 预热，并从 1.0×10^{-5} 余弦衰减至 5.0×10^{-6} 。与中期训练类似，我们冻结 MoE 路由器权重并禁用 EP 组均衡损失。SFT 训练使用 MTP 损失权重 0.1、全局 batch size 32、全局序列长度 128k。在 RoPE [83] 上，我们保持 $\theta_{SWA} = 10,000$ ，并将 $\theta_{Full} = 5,000,000$ 以适配 128k 上下文长度 [84]。

5.2. 可扩展 RL

在 LLM 的 RL 中，我们优化策略 π_θ 以最大化轨迹 $\tau = (s_0, a_0, \dots, s_T)$ 上的终止奖励，其中 a_t 为状态 s_t 生成的 token。然而在推理任务中，该过程会因高梯度方差而产生严重不稳定，并被极长序列与模型规模进一步放大（图 5 (2)）。方差主要来自高吞吐推理引擎与训练框架之间的基础设施差异，以及迭代更新固有的离策略失配。在此设置下，重要性采样本身不稳定，微小的 token 概率偏移会累积为噪声梯度，阻碍收敛。

5.2.1. MIS 过滤的策略优化 (MIS-PO)

为应对这些稳定性挑战，我们提出 MIS-PO，受 Metropolis Independence Sampling (MIS) [39, 40] 启发。我们将推理策略视为提议分布、训练策略视为目标分布，将更新限制在与目标分布足够接近的样本上。与通过有界比率缩放梯度且常伴随高方差的重要性采样不同，MIS-PO 对分布外样本施加二值掩码过滤，并将保留轨迹视为近似 on-policy，从而显著降低梯度方差并稳定优化。

形式化地，我们定义二值指示函数 $\mathbb{I}(x) = \mathbb{1}[\rho_{\min} \leq x \leq \rho_{\max}]$ ，并在两个粒度上使用。**token** 级上，用其过滤概率比 $x_t = \pi_{\theta_{\text{old}}}(a_t|s_t)/\pi_{\theta_{\text{vllm}}}(a_t|s_t)$ ，以抑制训练与推理策略的局部不匹配 [37]。轨迹级上，将同一指示函数应用于几何均值比 $\bar{\rho}(\tau) = (\prod_t x_t)^{\frac{1}{T}}$ ，从而丢弃偏离目标分布较大的整条轨迹。新的 actor 损失以双层离散掩码替代连续重要性权重：

$$\mathcal{L}_{\text{actor}} = -\mathbb{E}_{\tau \sim \pi_{\theta_{\text{vllm}}}} [\mathbb{I}(x_t) \cdot \mathbb{I}(\bar{\rho}(\tau)) \cdot \log \pi_\theta(a_t|s_t) \cdot \hat{A}_t]. \quad (2)$$

将有效样本视为近似 on-policy 后，该目标在信任区域约束下显著降低长程推理任务的梯度方差。图 5

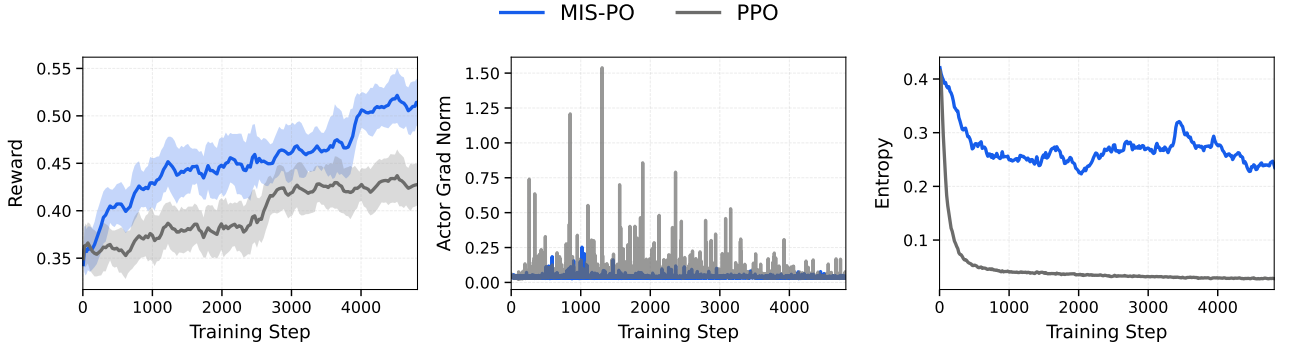


Figure 5: **MIS-PO** 与 **PPO** 在内部模型上的可扩展性对比。(1) 效率: MIS-PO 具有更高样本效率, 以更快的收敛趋势达到更高奖励平台。(2) 稳定性: MIS-PO 明显稳定训练动态, 抑制梯度噪声并消除策略梯度范数的大幅尖峰。(3) 探索保持: MIS-PO 的熵衰减更慢, 有助于更好的探索—利用平衡。

给出约 5,000 训练步的消融结果, MIS-PO 的 actor 梯度范数噪声显著低于 PPO, 表明其可扩展性更好。更多消融见附录 D.2.3。

为进一步稳定训练动态, 我们采用多项技术: 截断感知的价值自举 [87] 用于纠正上下文截断带来的乐观奖励偏差, 以及路由置信度监控以预测 MoE 架构特有的不稳定性。

截断感知的价值自举。对上下文截断轨迹赋予零奖励会混淆“截断”与“任务失败”, 无法区分未完成与错误结果, 从而惩罚长链条推理。为此, 我们将零奖励替换为终止状态的自举价值估计, 等价于把截断视为视野中断而非终止失败。轨迹 τ_i 的修正奖励定义为:

$$\hat{R}_i = \begin{cases} V_\phi(s_T) & \text{若响应被截断,} \\ R_i & \text{否则。} \end{cases} \quad (3)$$

实证表明, 该截断感知自举即使在 20% 的截断率下也能稳定训练, 避免不完整轨迹常引发的奖励退化 [88, 89]。消融实验验证该技术对竞赛级基准尤为有效, 因为长程推理使截断效应更常见。

路由置信度作为稳定性代理。近期研究 [36, 38] 将 RL 稳定性与 MoE 路由一致性关联起来。在此基础上, 我们提出路由置信度 (Σ_k) 作为稳定性代理, 它是被激活专家的平均概率质量。较低的 Σ_k 意味着更高的路由不确定性, 会放大训练—推理失配。通过初步实验, 我们发现明显的稳定性相变: 低路由置信度的模型更脆弱, 需要极端稳定化 (例如, Router Replay [1, 36, 38]、严格 on-policy 更新 [90])。相比之下, 高路由置信度的模型更为稳健, 可在无需复杂干预的情况下进行离策略训练。

RL 训练动态。 为全面展示方法, 我们在图 6 中给出带可验证奖励的 RL (RLVR) 训练动态及 Step 3.5 Flash 的下游评测提升。训练奖励稳步上升, 表明学习过程稳定有效。此外, Step 3.5 Flash 在多种基准上取得一致提升: IMO-AnswerBench [91] +3.2%, CF-Div2-Stepfun-cpp (附录 E.2.1: 自定义 CodeForces² Div.2 基准) +6.1%, ARC-AGI-1 [92] +10.6%, HLE_{text} [93] +3.4%。

5.2.2. 奖励系统

我们将 RL 框架解耦为可验证奖励的 RL (RLVR [94]) 与不可验证奖励的 RL (例如 RLHF [95]), 分别配备与监督特性匹配的奖励模型。

²<https://codeforces.com/>

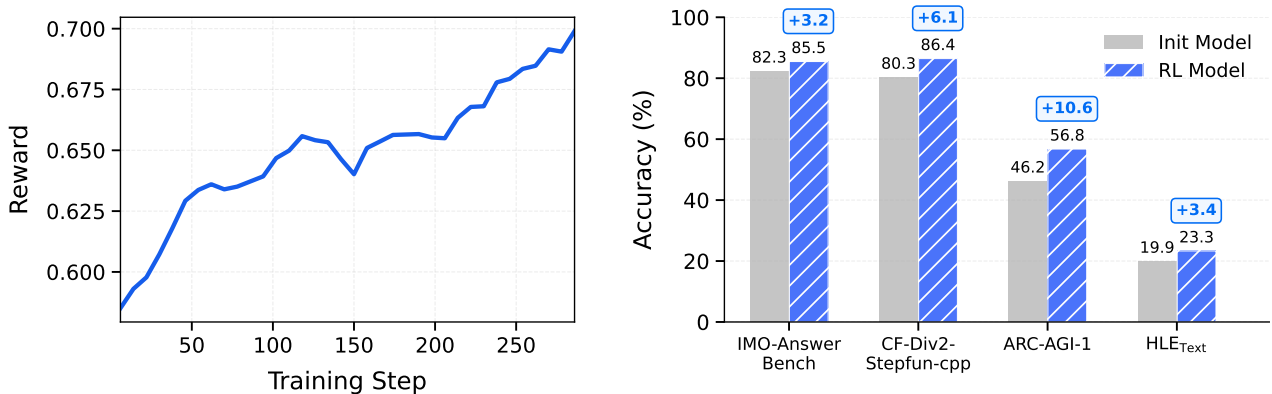


Figure 6: **Step 3.5 Flash** 的 RL 训练动态与跨域提升。RL 使奖励稳步增长（左），并在多项基准上带来一致的准确率提升（右）。

可验证奖励。在 RLVR 中，每个提示都配有任务特定验证器输出奖励。基于规则的检查器用于逻辑、指令遵循与代码任务，基于模型的验证器用于 STEM 任务。在内部模型上进行 450 步 RL 训练的消融实验表明，STEM 任务使用模型验证器比直接的数学验证平均高 2.0%；更多细节见附录 D.2.2。

不可验证奖励。对不可验证任务，我们使用成对生成式奖励模型（GenRM [96]），将回答与固定参考进行对比。GenRM 为推理模型，输出回答获胜的置信度分数。该分数随后被转换为 Bradley-Terry 胜率 [97] 作为奖励信号。长度控制在 GenRM 内建模为置信度惩罚，并传递到胜率奖励，从而抑制 RL 训练中的过度长度增长。我们进一步通过对伪造引用、过度自信陈述或语言不一致的回答赋零奖励，提升鲁棒性。

智能体奖励。搜索任务使用基于实体匹配分数的 LLM 进行评估。在报告生成中，基于 rubric 的 LLM 裁判会评估研究问题、rubric 规格与候选报告，输出三值判断（满足、部分满足、不满足）[98]。由于中间类别常与专家偏好不一致，我们将其映射为非对称二值奖励，以获得更清晰的学习信号并加速向专家偏好对齐的收敛。

GenRM 训练与 MetaRM。 我们通过使用 RM 专用提示对 SFT 模型进行微调来初始化 GenRM。在 RL 训练中，使用整理后的成对偏好数据，并采用与标量奖励模型类似的 logsigmoid 损失。为提升 GenRM 鲁棒性，我们引入 MetaRM（额外验证器）以惩罚含有伪推理（即以错误逻辑得到正确偏好）的回答，从而在检测到此类模式时降低训练奖励。在内部模型上跨 200 步 RL 训练的消融实验中，加入 MetaRM 的 GenRM 在所有基准上均比原始 GenRM 高 0.5%–3%。

5.2.3. 超参数

在 rollout 中，我们将采样温度与 top- p 均设为 1.0，最大序列长度 128k token。每次生成：推理任务采样 256 个不同提示，每个提示 16 个响应；人类偏好任务采样 512 个提示，每个提示 8 个响应；工具使用任务采样 128 个提示，每个提示 8 个响应。rollout 后将完成样本划分为 mini-batch，并以单 epoch 训练：actor 使用 4 个 mini-batch，critic 使用 12 个 mini-batch。优化采用 Muon，权重衰减 0.1。actor 学习率 2×10^{-6} ，预热 20 步；critic 学习率 5×10^{-6} ，预热 50 步。遵循 ORZ [90]，我们设置 γ 与 λ 均为 1。最后阶段使用无偏 KL 损失 [85]，系数 0.001。对公式 (2)，token 级与轨迹级掩码区间分别设为 [0.5, 2] 与 [0.996, 1.001]。

领域	样本数	Token	语料占比
数学	68055	0.98B	11.19%
代码	86421	1.23B	21.10%
STEM	120399	0.55B	6.31%
逻辑	93323	0.81B	13.87%
通用	314495	0.80B	9.16%
代码智能体	37240	0.90B	17.70%
工具使用	114507	0.76B	8.72%
搜索智能体	20256	0.50B	8.75%
长上下文	15565	0.70B	4.00%
总计	870687	7.23B	100.00%

Table 3: 第一阶段 SFT 的数据统计。

5.3. 数据合成与整理

我们通过聚合开源数据、合成生成与用户轨迹，构建多样且难度均衡的提示池。采用统一的合成与整理流水线，将严格的全局过滤与领域特化精炼结合，以最大化推理密度。数据质量通过规则启发式与模型一致性检查的混合方式保障。最终数据集包含 871k 样本（7.23B token），详细统计见表 3。

5.3.1. 通用与推理

我们的训练语料汇集了社区提示、专家回答与多源开源合成数据，包括 数学 [90, 99–110]、编程 [111–113]，以及 科学与开放式问答 [114–117]。为最大化推理密度，我们采用统一流水线，将严格的全局过滤与领域特化精炼结合，并通过规则启发式与模型一致性检查的混合方式保障质量。具体而言，在数学中，我们通过专家引导的拒绝采样与大数算术合成来保证数值稳定；在编程中，我们优先选择严格的算法挑战以确保离线可执行性，并严格清除与 RAG 相关的幻觉。尤其是，我们抑制模型虚假声称可访问外部搜索引擎或伪装检索在线解的倾向。此外，我们将科学数据限制为答案明确、可唯一确定的问题。

为在实际场景中增强泛化，我们扩展了开源检查器³，并为样本加入若干真实世界约束。同时，我们从开源、合成与用户轨迹中收集通用提示，形成多样且难度均衡的池。该流程产出包含数百万样本、规模达十亿 token 的高保真数据集。

5.3.2. 通用化工具学习

我们提出一种以执行为驱动的数据生成框架，用于学习可靠的工具使用行为，解决现有合成流水线的数据不一致、不可验证与模型幻觉等关键问题。不同于随机探索 [118, 119] 或模型模拟 [5, 120]，我们将工具使用行为分解为原子意图，并用有限状态机（FSM）建模，显式分离抽象的工具调用逻辑与参数化执行约束。数据通过“采样—执行—验证”的拒绝采样闭环生成：所有候选轨迹均在真实环境中执行，并由确定性反馈验证，从而确保真实性并消除幻觉。通过组合式合成原子意图，该框架可规模化生成复杂、可控的工具使用场景。基于该范式，我们构建了超过 10 万条高质量轨迹，总计数十亿 token，为工具规划、推理与执行提供精确监督。

5.3.3. 代码智能体

代码智能体可通过可验证的环境构建与解答生成之间的闭环干预实现自我提升，执行反馈会持续改进两项能力。我们将环境构建与修复 bug、实现特性并列为一等能力，并在可验证奖励信号下进行合成。

³https://github.com/allenai/open-instruct/tree/main/open_instruct/IFEvalG

为此，我们构建了从 SWE-factory [121] 演化而来的专用智能体流水线，包含跨任务记忆池（检索历史构建成功作为 few-shot 示范）与循环检测机制以避免冗余探索。该流水线实现了 40% 的环境构建成功率，并通过包含 shell 命令与错误恢复的构建轨迹提供密集监督，形成模型自我演化的正反馈回路。为进一步提升信号质量，我们对环境构建轨迹进行规范化，抽象并掩盖与最终解决无关的短暂失败与冗余执行模式。这些自举环境作为动态测试床，结合执行反馈与单元测试生成高质量合成数据与奖励信号以进行持续对齐。实证显示存在双向迁移：构建专长加速编码性能，而在这些环境中编码又进一步提升构建准确率，如 DockSmith [122] 所示。借助该演化流水线，我们整理了 50k 个已验证环境，覆盖 15k+ GitHub 仓库与 20+ 编程语言。该多样化集合覆盖广泛的真实场景，为训练通用代码智能体提供坚实基础。此外，我们还纳入多项知名开源环境，包括 SWE-smith [123]、SWE-Gym [124]、R2E-Gym [125]、SWE-rebench [126] 与 SETA [127]。

5.3.4. 搜索与研究智能体

为促进高级信息检索，我们的流水线引入图结构与多文档综合以强化多跳推理。通过对知识图谱（例如 Wikidata5m [128]）进行拓扑扩展并模拟跨网站浏览轨迹，我们生成反映真实研究复杂性的样本。关键在于，为保证外部检索的必要性，我们使用 DeepSeek-R1 [129] 对生成查询进行校验，系统性地剔除无需工具即可由强推理模型解答的实例。随后，轨迹通过结构化报告生成流水线 [98] 进行精炼，确保严格指令遵循与结构完整性。具体而言，我们强制遵循预设研究计划，丢弃任何偏离结构的轨迹。随后对有效输出使用基于模型的判别器与启发式规则进行迭代清理，以解决非正式写作、时间幻觉与中英混杂等细粒度问题。该端到端方法在 RESEARCHRUBRICS [21] 基准上达到业界领先表现。

5.4. 智能体基础设施

推理与工具使用模板设计。为将推理与智能体能力有效整合到单一基础模型中，必须确定合适的思考过程与工具使用模板。针对推理模板，我们评估了三种管理策略。每轮都丢弃推理历史 [129] 虽鼓励独立生成，但在长时程任务中会导致失败（例如超过 100 轮的编码会话）。相反，保留完整推理历史会造成极高上下文消耗，迅速耗尽模型容量并阻塞后续工具调用。为此，我们采用选择性保留策略：仅保留由最近一条用户指令触发的工具使用轨迹中的推理痕迹。该设计在推理连贯性与上下文效率之间取得最优权衡，与近期前沿模型实践一致 [85, 130]。对于工具使用模板，我们比较了常见的 JSON 与 XML 格式。JSON 的严格语法（转义与分隔符）在小模型或训练不足时容易导致解析错误；而 XML 允许扁平字符串输出，语法开销显著更低。因此，我们选择 XML 以确保复杂现实场景下智能体编码的鲁棒性。

可扩展代码智能体基础设施。我们的集成架构聚焦于可扩展会话管理与跨框架泛化，以支持高吞吐智能体编码。核心是自研 Session-Router，通过 Kubernetes 协调容器生命周期，并借助 Tmux 保持交互一致性。该架构支持数千并发环境的无缝状态持久化，无需为每个脚手架手工配置 Docker。为确保在多样智能体工作流中的高泛化，我们训练模型适配广泛交互框架，从学术标准（例如 OpenHands [131]、SWE-agent [132]、Terminus-2 [16]）到企业级协议（例如 Kilocode [133]、Roocode [134]、ClaudeCode [135]）。在训练中暴露于这些不同交互范式，有效防止模型过拟合特定流水线模式，使其在不同执行环境下均保持稳健。

6. 评测

6.1. 预训练评测

评测设置。我们在一系列覆盖不同能力的基准上评估 Step 3.5 Flash：(1) 通用语言理解与推理，包括 BBH [136]、MMLU [137]、MMLU-Redux [138]、MMLU-Pro [139]、HellaSwag [140]、WinoGrande [141]、GPQA [142]、SuperGPQA [143] 与 SimpleQA [144]。(2) 数学推理，包括 GSM8K [145] 与 MATH [146]。

extbf 基准	示例数	Step 3.5 Flash Base	MiMo-V2 Flash Base	GLM-4.5 Base	DeepSeek V3.1 Base	DeepSeek V3.2 Exp Base	Kimi-K2 Base
激活参数量	-	11B	15B	32B	37B	37B	32B
总参数量	-	196B	309B	355B	671B	671B	1043B
通用							
BBH	3-shot	88.2	88.5	86.2	88.2†	88.7†	88.7
MMLU	5-shot	85.8	86.7	86.1	87.4†	87.8†	87.8
MMLU-Redux	5-shot	89.2	90.6	-	90.0†	90.4†	90.2
MMLU-Pro	5-shot	62.3	73.2	-	58.8†	62.1†	69.2
HellaSwag	10-shot	90.2	88.5	87.1	89.2†	89.4†	94.6
WinoGrande	5-shot	79.1	83.8	-	85.9†	85.6†	85.3
GPQA	5-shot	41.7	43.5*	33.5*	43.1*	37.3*	43.1*
SuperGPQA	5-shot	41.0	41.1	-	42.3†	43.6†	44.7
SimpleQA	5-shot	31.6	20.6	30.0	26.3†	27.0†	35.3
数学							
GSM8K	8-shot	88.2	92.3	87.6	91.4†	91.1†	92.1
MATH	4-shot	66.8	71.0	62.6	62.6†	62.5†	70.2
代码							
HumanEval	3-shot	81.1	77.4*	79.8*	72.5*	67.7*	84.8*
MBPP	3-shot	79.4	81.0*	81.6*	74.6*	75.6*	89.0*
HumanEval+	0-shot	72.0	70.7	-	64.6†	67.7†	-
MBPP+	0-shot	70.6	71.4	-	72.2†	69.8†	-
MultiPL-E HumanEval	0-shot	67.7	59.5	-	45.9†	45.7†	60.5
MultiPL-E MBPP	0-shot	58.0	56.7	-	52.5†	50.6†	58.8
中文							
C-EVAL	5-shot	89.6	87.9	86.9	90.0†	91.0†	92.5
CMMLU	5-shot	88.9	87.4	-	88.8†	88.9†	90.9
C-SimpleQA	5-shot	63.2	61.5	70.1	70.9†	68.0†	77.6

Table 4: 预训练评测结果。* 表示原始分数不可用；为公平比较，我们在与 Step 3.5 Flash 相同的测试条件下复现并报告结果。† 表示 DeepSeek 分数引用自 MiMo-V2-Flash 报告 [30]。

- (3) 编码能力，包括 HumanEval [147]、MBPP [148]、HumanEval+、MBPP+ [149] 与 MultiPL-E [150]。
(4) 中文理解，包括 C-Eval [151]、CMMLU [152] 与 C-SimpleQA [153]。

评测结果。表 4 总结了 Step 3.5 Flash 在通用推理、数学、代码与中文基准上的预训练评测。尽管仅激活 11B 参数（总参数 196B），Step 3.5 Flash 仍与显著更大的稀疏基线（激活 15–37B；总参数 309–1043B）保持广泛竞争力，体现了强劲的准确率—效率权衡。在核心通用基准上，Step 3.5 Flash 在 BBH 上达到 88.2（距最佳仅 0.5），在 MMLU 上达到 85.8。值得注意的是，Step 3.5 Flash 在 SimpleQA 上达到 31.6，超过 DeepSeek-V3.2-Exp Base（27.0），且总参数仅 196B，相比 671B（约 ~3.4×），显示出更高的参数效率密度。Step 3.5 Flash 还展现出强大的编码能力：HumanEval 81.1、MultiPL-E HumanEval 67.7、MultiPL-E MBPP 58.0。总体而言，这些结果表明 Step 3.5 Flash 在单位激活计算下具备高性能，为下游推理与智能体后训练提供坚实基础。

6.2. 后训练评测

我们在代表性基准上评估 Step 3.5 Flash，包括推理导向的 HLE（文本子集）[154]、MMLU-Pro [139]、GPQA-Diamond [142]、AIME2025 [7]、HMMT [8]、IMO-AnswerBench [91]；代码相关的 LiveCodeBench-v6 (2024.08–2025.05) [9]、CF-Div2-Stepfun⁴、SWE-Bench Verified [13] 与 SWE-Bench

⁴<https://huggingface.co/datasets/stepfun-ai/CF-Div2-Stepfun>

Multilingual [14]; 智能体系列 τ^2 -Bench [15]、Terminal-Bench 2.0 [16]、GAIA [19]、BrowseComp [17]、xbench-DeepSearch [20]、BrowseComp-zh [18] 与 RESEARCHRUBRICS [21]; 通用相关的 Arena-Hard v2 [155]、IFBench [156] 与 MultiChallenge [157]; 以及长上下文相关的 LongBench v2 [158]、MRCR [159]⁵、FRAMES [160] 与 RepoQA [161]。

我们还采用 并行协同推理 (**PaCoRe**) 范式 [42], 在推理、通用与长上下文基准上研究 Step 3.5 Flash 的测试时扩展特性。依托 Step 3.5 Flash 的极致推理效率, 该方法通过并行推理轨迹并在多轮协调中综合其洞见, 将推理能力与上下文限制解耦。具体而言, 我们采用多轮 PaCoRe 轨迹配置 $\vec{K} = [4, 4, 4, 4]$, 在多个基准上获得显著增益。

我们保持最大序列长度 256k, 使用默认解码配置 (温度与 top-p 均为 1.0)。在原始 128k 位置嵌入之上, 我们应用 YaRN [162], 缩放因子为 2.0, 并仅作用于全注意力层。所有方法均报告 pass@1 准确率, 基于每题多次独立生成的平均表现: AIME 2025、HMMT 2025 Feb. 与 HMMT 2025 Nov. 为 64 次; IMO-AnswerBench、LiveCodeBench、GPQA-Diamond 与 MultiChallenge 为 8 次; HLE 为 1 次, 其余基准为 4 次。更多细节见附录 E。

评测结果。表 5 给出了 Step 3.5 Flash 在推理、代码智能体、通用智能体、长上下文理解与通用能力等基准上的全面对比。尽管仅激活 11B 参数 (总参数 196B), Step 3.5 Flash 仍在广泛任务上表现强劲, 尤其在 AIME 2025、HMMT 2025 Feb.、HMMT 2025 Nov.、IMO-AnswerBench 与 LiveCodeBench-v6 等高强度推理基准上突出。它持续优于参数更多的开源模型, 并达到 GPT-5.2 xHigh 与 Gemini 3.0 Pro 等前沿模型的同等级表现。值得注意的是, Step 3.5 Flash 在智能体评测上也取得了强结果, 包括 SWE-Bench Verified、Terminal-Bench 2.0、BrowseComp (含上下文管理)、GAIA 与 τ^2 -Bench, 凸显其稳健的工具使用与长程决策能力。

7. 局限性

Token 效率。 Step 3.5 Flash 已达到前沿级智能, 但当前在达到与 Gemini 3.0 Pro 相当质量时仍需要更长的生成轨迹。下一步我们将对思维过程进行剪枝与压缩, 以提升效率并保持相同的竞争性表现。

高效的通用与专精统一。 我们希望将通用能力与深度领域专长统一。为高效实现这一目标, 我们正在推进 on-policy 蒸馏的变体, 使模型以更高样本效率内化专家行为。

面向开放世界智能体任务的 RL。 尽管 Step 3.5 Flash 在学术智能体基准上表现有竞争力, 但下一阶段的智能体 AI 需要将 RL 应用于专业工作、先进工程与科学研究中的复杂专家级任务。解决这些挑战是部署具备真正自主能力的智能体的前提。

适用范围与约束。 Step 3.5 Flash 面向编码与工作型任务优化, 但在分布迁移时稳定性可能下降。这通常发生在高度专业化领域或长程多轮对话中, 模型可能出现重复推理、中英混杂输出, 或在时间与身份认知上产生不一致。

贡献者

作者名单按英文名首字母顺序排列。

⁵<https://huggingface.co/datasets/openai/mrcr>

基准	Step 3.5 Flash		MiniMax M2.1	MiMo V2 Flash	GLM 4.7	DeepSeek V3.2	Kimi K2.5	Gemini 3.0 Pro	Claude Opus 4.5	GPT-5.2 xHigh
	原始	PaCoRe								
激活参数量	11B		10B	15B	32B	37B	32B	-	-	-
总参数量	196B		230B	309B	355B	671B	1T	-	-	-
推理										
AIME 2025	97.3	99.9	83.0	95.1*	95.7	93.1	96.1	95.0	92.8	100.0
HMMT 2025 Feb.	98.4	100.0	71.0*	95.4*	97.1	92.5	95.4	97.5†	92.9†	99.4
HMMT 2025 Nov.	94.0	97.8	74.3*	91.0*	93.5	90.2	91.1	94.5†	91.7*	97.1*
IMO-AnswerBench	85.4	88.8	60.4*	80.9*	82.0	78.3	81.8	83.3†	84.0†	86.3†
LiveCodeBench-v6	86.4	88.9	75.4*	81.6*	84.9	83.3	85.0	90.7†	84.8†	87.7†
CF-Div2-Stepfun-cpp	86.1	93.3	59.0*	46.9*	74.1*	81.6*	73.6*	83.5*	72.2*	-
MMLU-Pro	84.4	84.8	88.0	84.9	84.3	85.0	87.1	90.1†	89.5†	87.4†
GPQA-Diamond	83.5	85.0	83.0	84.1*	85.7	82.4	87.6	91.9	87.0	92.4
HLE _{text}	23.1	27.9	22.2	22.1	24.8	25.1	31.5	37.7†	30.8†	35.5†
代码智能体										
SWE Verified	74.4	-	74.0	73.4	73.8	73.1	76.8	76.2	80.9	80.0
SWE Multilingual	67.4	-	72.5	71.7	66.7	70.2	73.0	65.0†	77.5†	72.0†
Terminal-Bench 2.0	51.0	-	47.9	38.5	41.0	46.4	50.8	56.9†	59.3†	54.0†
通用智能体										
BrowseComp	51.6	-	47.4	45.4	52.0	51.4	60.6	37.8†	37.0†	-
BrowseComp (w. Ctx Manage)	69.0	-	62.0	58.3	67.5	67.6	74.9	59.2†	57.8†	65.8
BrowseComp-ZH	66.9	-	47.8*	51.2*	66.6	65.0	62.3*	66.8*	62.4*	76.1*
GAIA	84.5	-	64.3*	78.2*	61.9*	75.1*	75.9*	76.6*	76.1*	83.5*
xbench-DeepSearch-2505	83.7	-	68.7*	69.3*	72.0*	78.0*	76.7*	78.3*	77.0*	83.0*
xbench-DeepSearch-2510	56.3	-	43.0*	44.0*	52.3*	55.7*	40.0†	57.7*	59.3*	67.0*
RESEARCHRUBRICS	65.3	-	60.2*	54.3*	62.0*	55.8*	59.5*	50.1*	61.6*	57.8*
τ ² -Bench	88.2	-	86.6*	84.1*	87.4	85.2*	85.4*	90.7	92.5	85.5*
通用										
Arena-Hard-v2.0	74.0	93.1	63.1*	68.2*	73.1*	66.0*	85.8*	81.7†	76.7†	80.6†
MultiChallenge	55.7	60.8	50.5*	44.3*	67.8*	57.1*	73.6*	71.8*	65.8*	71.9*
IFBench	67.4	56.8	70.0	64.0†	68.0†	61.0†	72.8*	70.4†	58.0†	75.4†
长上下文										
LongBench v2	57.5	62.0	53.9*	60.6†	59.1*	58.4†	61.0	70.0*	67.8*	62.4*
MRCR-Sneedle	28.8	26.3	20.0†	19.9†	25.4†	27.2†	36.5*	73.0†	54.0*	88.2*
FRAMES-Oracle	76.5	77.2	76.5*	78.0*	75.1*	80.1*	77.4*	79.7*	85.8*	87.3*
RepoQA	88.5	88.7	88.2*	91.2*	89.5*	91.9*	89.8*	91.5*	95.7*	93.8*

Table 5: Step 3.5 Flash 与闭源/开源模型的对比。* 表示原始分数不可用或低于我们复现结果；为公平比较，我们在与 Step 3.5 Flash 相同的测试条件下复现并报告结果。† 表示分数来源于非官方渠道，包括技术报告或独立评测平台。我们对 HLE 的评测仅覆盖文本子集。BrowseComp (w. Ctx Manage) 表示启用上下文管理的 BrowseComp 评测。

Ailin Huang	Chen Xu	Gulin Yan	Hongyu Zhou
Ang Li	Chenfeng Yu	Guoliang Cao	Hongyuan Wang
Aobo Kong	Chengting Feng	Guopeng Li	Houyong Chen
Bin Wang	Chengyuan Yao	Han Cheng	Huangxi Zhu
Binxing Jiao	Chunrui Han	Hangyu Guo	Huimin Wu
Bo Dong	Dan Ma	Hanshan Zhang	Huiyong Guo
Bojun Wang	Dapeng Shi	Hao Nie	Jia Wang
Boyu Chen	Daxin Jiang	Haonan Jia	Jian Zhou
Brian Li	Dehua Ma	Haoran Lv	Jianjian Sun
Buyun Ma	Deshan Sun	Hebin Zhou	Jiaoren Wu
Chang Su	Di Qi	Hekun Lv	Jiaran Zhang
Changxin Miao	Enle Liu	Heng Wang	Jiashu Lv
Changyi Wan	Fajie Zhang	Heung-Yeung Shum	Jiashuo Liu
Chao Lou	Fanqi Wan	Hongbo Huang	Jiayi Fu
Chen Hu	Guanzhe Huang	Hongbo Peng	Jiayu Liu

Jie Cheng	Qi Han	Xiangwen Kong	Yongchi Zhao
Jie Luo	Qian Zhao	Xiangyu Liu	Yongshen Long
Jie Yang	Qinglin He	Xiangyu Zhang	Yongyao Wang
Jie Zhou	Qinxin Du	Xiaobo Yang	Yousong Guan
Jieyi Hou	Qiuping Wu	Xiaojia Liu	Yu Zhou
Jing Bai	Quan Sun	Xiaolan Yuan	Yuang Peng
Jingcheng Hu	Rongqiu Yang	Xiaoran Jiao	Yuanhao Ding
Jingjing Xie	Ruihang Miao	Xiaoxiao Ren	Yuantao Fan
Jingwei Wu	Ruixin Han	Xiaoyun Zhang	Yuanzhen Yang
Jingyang Zhang	Ruosi Wan	Xin Li	Yuchu Luo
Jishi Zhou	Ruyan Guo	Xin Liu	Yudi Zhao
Junfeng Liu	Shan Wang	Xin Wu	Yue Peng
Junzhe Lin	Shaoliang Pang	Xing Chen	Yueqiang Lin
Ka Man Lo	Shaowen Yang	Xingping Yang	Yufan Lu
Kai Liang	Shengjie Fan	Xinran Wang	Yuling Zhao
Kaibo Liu	Shijie Shang	Xu Zhao	Yunzhou Ju
Kaijun Tan	Shiliang Yang	Xuan He	Yurong Zhang
Kaiwen Yan	Shiwei Li	Xuanti Feng	Yusheng Li
Kaixiang Li	Shuangshuang Tian	Xuedan Cai	Yuxiang Yang
Kang An	Siqi Liu	Xuqiang Zhou	Yuyang Chen
Kangheng Lin	Siye Wu	Yanbo Yu	Yuzhu Cai
Lei Yang	Siyu Chen	Yang Li	Zeja Weng
Liang Lv	Song Yuan	Yang Xu	Zetao Hong
Liang Zhao	Tiancheng Cao	Yanlin Lai	Zexi Li
Liangyu Chen	Tianchi Yue	Yanming Xu	Zhe Xie
Lieyu Shi	Tianhao Cheng	Yaoyu Wang	Zheng Ge
Liguo Tan	Tianning Li	Yeqing Shen	Zheng Gong
Lin Lin	Tingdan Luo	Yibo Zhu	Zheng Zeng
Lina Chen	Wang You	Yichen Lv	Zhenyi Lu
Luck Ma	Wei Ji	Yicheng Cao	Zhewei Huang
Mengqiang Ren	Wei Yuan	Yifeng Gong	Zhichao Chang
Michael Li	Wei Zhang	Yijing Yang	Zhiguo Huang
Ming Li	Weibo Wu	Yikun Yang	Zhiheng Hu
Mingliang Li	Weihao Xie	Yin Zhao	Zidong Yang
Mingming Zhang	Wen Sun	Yingxiu Zhao	Zili Wang
Mingrui Chen	Wenjin Deng	Yinmin Zhang	Ziqi Ren
Mitt Huang	Wenzhen Zheng	Yitong Zhang	Zixin Zhang
Na Wang	Wuxun Xie	Yixuan Zhang	Zixuan Wang
Peng Liu	Xiangfeng Wang	Yiyang Chen	

Appendix

A. Architecture Details

Table 6 summarizes key architecture hyper-parameters of Step 3.5 Flash.

Hyper-Parameter	Value
Backbone	
Vocabulary size (V)	128,896
Model width (d_{model})	4096
Transformer blocks	45 (3 dense + 42 MoE)
MoE FFN	
Experts per MoE block	288 + 1 shared
Routing	top- $k = 8$
Dense FFN hidden size	11,264
MoE expert hidden size	1,280
Attention	
Hybrid block structure	3 SWA blocks + 1 full attention block
SWA window size	512
KV heads (GQA)	8
Query heads (full / SWA)	64 / 96
Gate Type	head-wise on output
Head dimension	128
RoPE θ	10,000
RoPE dims (full / SWA)	64 / 128
Multi-Token Prediction	
MTP blocks	3 (Dense SWA)
Parameter Counts	
Total params (backbone)	196B
Activated params / token (backbone)	11B
Total params (with MTP3)	198B
Activated params / token (with MTP3)	13B

Table 6: Key architecture hyper-parameters of Step 3.5 Flash. “Activated params” are reported per token and exclude embedding/output matrices.

A.1. Head-wise Gated Attention

Each attention head is assigned a lightweight, input-dependent scalar gate, allowing the model to dynamically modulate information flow across the hybrid layout with negligible computational overhead.

Formally, for a (single) head of dimension d , let $\mathbf{q}_i, \mathbf{k}_j, \mathbf{v}_j \in \mathbb{R}^d$ denote the query vector at position i and the key and value vectors at position j , the scaled dot-product scores s , the corresponding attention weights α and the outputs $\mathbf{y}$ are computed as follows:

$$s_{i,j} = \langle \mathbf{q}_i, \mathbf{k}_j \rangle / \sqrt{d}, \quad Z_i = \sum_{j'} \exp(s_{i,j'}), \quad \alpha_{i,j} = \exp(s_{i,j}) / Z_i, \quad \mathbf{y}_i = \sum_j \alpha_{i,j} \mathbf{v}_j. \quad (4)$$

Given the input representation $\mathbf{x}_i$ at position i , we compute a head-wise gate g_i to modulate the head

output:

$$g_i = \sigma(\mathbf{w}_{gate}^\top \mathbf{x}_i), \quad o_i^{gate} = g_i y_i, \quad (5)$$

where $\sigma(\cdot)$ is the sigmoid function and $\mathbf{w}_{gate}$ is a learnable vector.

Head-wise gated attention can be viewed as introducing an *input-dependent sink token* [33] into the attention mechanism. Substituting $\sigma(g) = \frac{1}{1+\exp(-g)}$ into Equation 5, we have

$$o_i^{gate} = \sum_j \frac{\exp(s_{i,j})}{Z_i + e^{-g_i} Z_i} \mathbf{v}_j, \quad (6)$$

where $\exp(-g_i)Z_i$ acts as an *input-dependent sink mass* in the softmax normalizer. As shown in Section 2.3, this adaptive formulation consistently outperforms fixed (input-independent) sink tokens.

A.2. Speed Benchmark of Attention Enhancements

We conduct simulations with MTP-3 to evaluate the latency overheads of the two enhancements under an ideal workload. Table 7 presents the relative increment of theoretical FLOPs and latency. Increasing the number of query heads in SWA slightly raises the FLOPs but has less impact on latency. This is due to a query-to- kv ratio of 12, which keeps SWA in the IO-bound region, even when considering MTP-3. For head-wise gating, neither FLOPs nor latency has noticeable difference because of its lightweight.

Backbone	SWA Heads	Setting	Decode (FLOPs / Lat.)		Prefill (FLOPs / Lat.)	
			64k	256k	64k	256k
Step 3.5 Flash (S3F1 layout)	64	no gate	1.00 / 1.00	1.00 / 1.00	1.00 / 1.00	1.00 / 1.00
	96	no gate	1.02 / 1.01	1.01 / 1.00	1.08 / 1.06	1.04 / 1.03
	64	head-wise	1.00 / 1.00	1.00 / 1.00	1.00 / 1.02	1.00 / 1.01
	96	head-wise	1.02 / 1.02	1.01 / 1.00	1.08 / 1.08	1.04 / 1.05

Table 7: Relative increment under different SWA head counts and gating strategies. The metrics are presented as FLOPs / Latency. The baseline configuration (first line) is normalized to 1.0.

Backbone	Layout	SWA Heads	Decode		Prefill	
			64K	256K	64K	256K
Step 3.5 Flash	S3F1	64	1.00	1.00	1.00	1.00
	S3F1+Head	96	1.02	1.01	1.08	1.04
	S1F1	64	1.18	1.47	1.38	1.71
	FFFF	64	1.51	2.33	2.07	3.00
Internal 30B-A3B	S3F1	32	1.00	1.00	1.00	1.00
	S3F1+Head	48	1.02	1.01	1.05	1.02
	S1F1	32	1.42	1.74	1.50	1.80
	FFFF	32	2.21	3.16	2.47	3.34

Table 8: Relative FLOPs cost across different backbones and attention patterns. The head count refers to SWA heads. For each backbone, the configuration with minimum FLOPs (S3F1 with reduced heads) is the baseline (1.0).

A.3. Meta Token

Recent literature [163–165] has shown both theoretically and empirically that pre-pending structured metadata to pre-training sequences can improve data efficiency and accelerate convergence: by exposing high-level attributes (*e.g.*, modality, language, domain), metadata provides global cues that reduce uncertainty about the upcoming content and thus makes next-token prediction easier.

Motivated by this paradigm, we associate each training example with a metadata string $\mathbf{M}$ in a human-readable format, including content type (*e.g.*, Code, Book, Paper, Web), language (*e.g.*, EN, ZH), domain, and source. We then prepend $\mathbf{M}$ to the original token sequence $\mathbf{x}$, forming a single training sequence $\mathbf{s} = [\mathbf{M}; \mathbf{x}]$. During pre-training, the model is trained to maximize the likelihood of $\mathbf{s}$:

$$\mathcal{L}_{\text{full}}(\theta) = - \sum_{t=1}^{|\mathbf{s}|} \log P_{\theta}(s_t \mid \mathbf{s}_{<t}). \quad (7)$$

After an initial phase of approximately 3.8T tokens, we keep $\mathbf{M}$ in the context but mask out its positions from the loss while continuing to predict the payload tokens:

$$\mathcal{L}_{\text{mask}}(\theta) = - \sum_{t=|\mathbf{M}|+1}^{|\mathbf{s}|} \log P_{\theta}(s_t \mid \mathbf{s}_{<t}) = - \sum_{t=1}^{|\mathbf{x}|} \log P_{\theta}(x_t \mid \mathbf{M}, \mathbf{x}_{<t}). \quad (8)$$

We hypothesize that by this stage the model has already learned to effectively use metadata as a conditioning signal. Masking the metadata loss therefore allocates optimization pressure entirely to the payload tokens, while still benefiting from the explicit conditioning on data characteristics.

A.4. Pre-training Ablations Details

We conduct controlled pre-training ablations to isolate the effects of (i) different hybrid attention layout and (ii) sink tokens versus head-wise gated attention.

Hybrid attention layout. We adopt a **30B-A3B MoE** architecture to evaluate the downstream impact of different hybrid attention layout under a fixed token budget. The training follows a strict, multi-stage pipeline: a 30B-token warmup phase, followed by 1T tokens of main pre-training, a 300B-token cooldown phase, and an additional 100B-token long-context specialization stage—totaling approximately 1.4T tokens. Supervised fine-tuning (SFT) is then performed on a $0.1\times$ downsampled dataset. Full training details are provided in Table 9.

Gate vs. sink (scaled setting). We pre-train a **100B-A10B MoE** model for $\sim 250\text{B}$ tokens to compare sink tokens and head-wise gating under a larger-scale regime.

Pre-training results of the architectural ablations are presented in Tables 2 and 10. We employ the evaluation protocols detailed in Section 6.1. Specifically, GPQA [142] is evaluated using 5-shot prompting, while HumanEval [166] and MBPP [167] utilize 3-shot prompting.

The post-training results in Table 1 are aggregated as follows:

- **Reasoning:** The average of MMLU-Pro [139], GPQA-Diamond [142], LiveCodeBench v6 [9], and LiveBench [168].
- **Math:** The average of AIME 2024 [169], AIME 2025 [170], HMMT 2025 Feb. [171], and CNMO 2024⁶.
- **Code:** The average of CF-Div2-Stepfun and LiveCodeBench v6 [9].

⁶<https://www.cms.org.cn/Home/comp/comp/cid/12.html>

Hyper-Parameter	100B-A10B	30B-A3B
Total Tokens	250B	1.4T
Optimizer	Muon [34]	
Peak learning rate	1.31×10^{-4}	1.1×10^{-3}
Batch-size warmup	-	First 30B tokens
Layers	43	48
Dimension	4096	2048
Leading Dense Layers	1	1
Routed Experts	96	128
Active Experts	4	8
Shared Experts	1	1
Load Balancing Method	Loss Free [64]	
Attention module	GQA8	
Sequence Length	4096	
Vocab Size	129280	
Batch Size	8192	16384
Weight Decay	0.1	
Partial RoPE	Disabled	Enabled
MTP	Enabled	Disabled

Table 9: Training configuration for the 100B-A10B and the 30B-A3B architecture ablation suite.

- **Sci:** Represented by GPQA-Diamond [142].
- **General:** The average of IFEval [172], IFBench [156], WildBench [173], Arena-Hard [155], and MultiChallenge [157].
- **LongCtx:** The average of six benchmark-level averages: (i) the average score across context lengths 8k-128k on RULER [174], (ii) the average score over the Short and Medium subsets of LongBench v2 [158], (iii) the average score across context lengths 8k-128k on HELMET [175], (iv) GSM-Infinite [176], (v) the overall score on FRAMES [160], and (vi) the overall score on RepoQA [161].

Tables 1 and 10 show that the vanilla *S3F1* layout underperforms the full-attention baseline on general pre-training benchmarks and consistently degrades SFT quality (*e.g.*, BBH: -4.3 ; SFT Avg: -0.7). Increasing the number of SWA query heads substantially closes this gap (*e.g.*, MMLU-Pro: $+3.7$; SFT Reasoning: $+0.4$), with only a minor regression on SFT Code (-0.6), while matching or exceeding the full-attention baseline on several metrics. Table 2 further demonstrates that head-wise gated attention yields an average improvement from 62.5 to 64.4 ($+1.9$) on the sink token metric.

B. Detail Analysis of Localized Activation Blow-up

To investigate the root cause of the localized activation blow-up, we analyze the tokens that trigger the largest expert activations across all layers, and identify two distinct large activation patterns: (1) Specific lexical items, such as special tokens and punctuation, commonly elicit large but not dramatic activations, particularly in the shallower layers. This pattern is not recognized as a failure mode by us,

Layout	SWA Heads	Pre-train Evaluation											
		BBH	MLLU	MLLU-Redux	MLLU-Pro	SimpleQA	GSM8K	MATH	HumanEval	MBPP	C-EVAL	CMMLU	Avg.
<i>FFFF</i>	32	66.0	64.5	69.7	35.7	7.2	70.0	39.2	48.8	53.4	69.7	70.5	54.1
<i>S1F1</i>	32	64.1	64.7	69.8	37.7	7.5	70.1	43.9	47.0	56.2	69.8	69.8	54.6
<i>S3F1</i>	32	61.7	64.2	69.4	33.7	8.0	67.4	41.5	47.6	56.0	69.5	70.9	53.6
<i>S3F1+Head</i>	48	65.3	65.9	71.0	37.4	7.5	72.2	44.5	48.8	58.6	70.2	71.0	55.7

Table 10: Pre-training evaluation results for hybrid attention layout ablations ($W=512$) on 30B-A3B. F denotes full attention and S denotes SWA; *S3F1* indicates three S and one F in the hybrid layout. *S3F1+Head* increases the number of SWA heads from 32 to 48.

as there is no rapid increment and it may serve as an internal mechanism for semantic modeling [60,177]. Another pattern is that (2) some high-frequency bi-grams trigger extremely large activations on the first token, which represents the failure mode we are investigating. The pattern is triggered by several factors: The frequency of a bi-gram’s occurrence is sufficiently high, and the MoE FFN is fine-grained enough, allowing an expert to specialize in that bi-gram without being regulated by the load balancing mechanism. This specialization serves as a shortcut: once the expert is activated, the output becomes deterministic, and other networks no longer influence the prediction. While finding shortcuts is a reasonable approach to minimizing loss, in a MoE model with a pre-norm architecture [76,77], there is a straightforward, pathological solution for achieving such deterministic predictions, as outlined next. The model’s final representation is the sum of the outputs from all layers, followed by a RMSNorm. This can be expressed as a combination of the outputs from the experts and the attention layers:

$$\mathbf{h}_{\text{final}} = \text{RMSNorm}(\underbrace{\text{expert}_{\text{outlier}}}_{\mathbf{h}_{\text{outlier}}} + \underbrace{\sum_{l=1}^L \text{attn}_l + \sum_{\substack{l,e \\ (l,e) \text{ is not a outlier}}} \text{expert}_{l,e}}_{\mathbf{h}_{\text{others}}}), \quad (9)$$

where attn , MoE, expert represent the output hidden states of their respective modules, while L and E denote the number of layers and experts, respectively. The straightforward solution is to boundlessly enlarge $\text{expert}_{\text{outlier}}$, then

$$\text{RMSNorm}(\mathbf{h}_{\text{final}}) = \lim_{c \rightarrow \infty} \text{RMSNorm}(c \cdot \hat{\mathbf{h}}_{\text{outlier}} + \mathbf{h}_{\text{others}}) = \text{RMSNorm}(\mathbf{h}_{\text{outlier}}), \quad (10)$$

where we decouple $\mathbf{h}_{\text{outlier}}$ to the magnitude c and the unit vector $\hat{\mathbf{h}}_{\text{outlier}}$ denoting the direction.

SwiGLU [78], the expert architecture in Step 3.5 Flash, provides a way to generate large outputs, even when the weight decay effectively suppresses the weight norms. SwiGLU is defined as follows:

$$\text{SwiGLU}(\mathbf{x}) = \mathbf{W}_{\text{down}} (\text{SiLU}(\mathbf{W}_{\text{gate}}\mathbf{x}) \cdot \mathbf{W}_{\text{up}}\mathbf{x}). \quad (11)$$

We analyze the activation norms of $\mathbf{W}_{\text{gate}}\mathbf{x}$ and $\mathbf{W}_{\text{up}}\mathbf{x}$ and find no significant differences between outlier experts and normal experts. However, the element-wise product produces abnormal outputs, which have

$$\|\text{SiLU}(\mathbf{W}_{\text{gate}}\mathbf{x})\| \cdot \|\mathbf{W}_{\text{up}}\mathbf{x}\| \approx \|\text{SiLU}(\mathbf{W}_{\text{gate}}\mathbf{x}) \cdot \mathbf{W}_{\text{up}}\mathbf{x}\|, \quad (12)$$

in outlier experts. It can be achieved only if $\text{SiLU}(\mathbf{W}_{\text{gate}}\mathbf{x})$ and $\mathbf{W}_{\text{up}}\mathbf{x}$ are highly aligned and concentrate on a very limited number of dimensions. Consequently, only a limited number of rows from $\mathbf{W}_{\text{up}}$ are utilized due to the sparse input. This observation leads us to prefer activation clipping over weight clipping, as activation’s numerical property directly contribute to the blow-up and the sparsity, and activation clipping can promptly address these issues. Besides, activation clipping has negligible negative effects, as well-behaved activations rarely exceed the threshold.

When using the Muon optimizer, gated linear units, such as SwiGLU, are susceptible to logit explosion. This vulnerability arises from similar mechanisms that cause explosion in attention, as reported in [5]. For an outlier expert specialized to some specific bi-gram, hidden states routed to it are expected to be closely aligned to its router embedding. We validate this by inputting the router embedding into a outlier expert and directly predicting outputs based on this expert’s output. The predicted distribution aligns with that of the real data and the entire network’s performance. Combined with the overly single training target (to predict the second token in the bi-gram), we argue that gradients w.r.t. the outlier expert’s parameters, $\mathbf{W}_{\text{gate}}$, $\mathbf{W}_{\text{up}}$ and $\mathbf{W}_{\text{down}}$, are not only abnormally low rank (denoted as r), but also consistently point in a direction that emphasizes the magnitude as analyzed in the first factor, without rotation. Let the update matrices of a parameter matrix $\mathbf{W} \in \mathbb{R}^{N \times N}$ to be

$$\Delta \mathbf{W} = \sum_i \sigma_i \mathbf{u}_i \mathbf{v}_i^\top = \underbrace{\sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^\top}_{\text{low rank signal}} + \underbrace{\sum_{j=r+1}^N \sigma_j \mathbf{u}_j \mathbf{v}_j^\top}_{\text{noise}} \quad (13)$$

Accumulating updates over optimization steps will rapidly increase the singular value of the low-rank signals, resulting in an explosion of the weight parameter. In the GLU structure, $\|\text{SiLU}(\mathbf{W}_{\text{gate}} \mathbf{x}) \cdot \mathbf{W}_{\text{up}} \mathbf{x}\|$ squares the spectral norm in our strong alignment case, making the progress more sharp. Additionally, Muon completely eliminates the influence of gradient magnitudes. During the blow-up process, RMSNorm reduces the gradients of large inputs. When using the Adam optimizer, its ϵ acts as a threshold to filter out small gradients during the learning rate adaptation, which can hinder the progress. In contrast, Muon consistently and effectively orthogonalizes the gradients, resulting in more aggressive updates.

C. Step Pre-training Data Foundation

C.1. Knowledge Data Construction

C.1.1. StepCrawl

Beyond standard web-scale datasets (*e.g.*, CommonCrawl), we develop **StepCrawl**, an in-house crawling and curation system designed to acquire *high-quality* and *diverse* tokens at scale. StepCrawl serves as a primary data source for both high-signal web pages and document-like content (notably PDFs), which frequently contain long-form, high-information-density material.

A key component of StepCrawl is a site and URL selection layer powered by a WebOrganizer-style model [178]. We adapt the capabilities introduced in WebOrganizer and further fine-tune a version tailored to our pipeline. During crawling, each fetched web page is analyzed by this model, forming a lightweight LM-in-the-loop feedback cycle that (i) filters SEO-driven and other low-utility pages, and (ii) guides crawl-budget allocation by balancing site categories (*e.g.*, preventing disproportionate crawling of tool and e-commerce sites) to preserve corpus diversity and reduce topical skew. In practice, StepCrawl processes on the order of $\sim 1\text{B}$ pages per day under this quality- and diversity-aware scheduling policy.

All crawling activities strictly adhere to `robots.txt` and site-specific access policies. The collected content is subsequently passed through a multi-stage filtering process (quality scoring, deduplication, and sanitization), ensuring that only high-utility and policy-compliant data are retained for training.

C.1.2. Quality Refinement and Stratification

Quality stratification. Inspired by Nemotron-CC [179]-style quality bucketing, we divide the internal web data into quality tiers and sample preferentially from higher tiers. We label each document using an ensemble of six lightweight scorers/classifiers and ensemble the tier assignments across scorers. In the final recipe, we keep High/Medium-High/Medium and discard Medium-Low/Low, which substantially improves token efficiency in ablations. For book and paper corpora, we apply the same stratification but restrict retention to High/Medium-High tiers exclusively during the annealing stage to maximize diversity. In addition to the shared six-scorer ensemble, we integrate additional domain-specific filters targeting STEM and knowledge-dense content, and down-sample overrepresented domains to ensure balanced representation.

Embedding-based cluster rebalancing. We leverage embedding-based corpus balancing as a principled way to further reduce redundancy and mitigate distribution skew. Specifically, we embed large-scale Chinese/English web data, run k-means clustering (100k+ clusters), and down-sample clusters with disproportionate mass. In ablations, this cluster-level rebalancing in the cooldown stage improves a broad set of benchmarks.

Knowledge-Intensive Mining and Augmentation. We construct a dedicated knowledge subset using a lightweight two-stage pipeline built on the shared embedding representation described above. First, a curated inventory of high-value entities, concepts, and relations is used to retrieve knowledge-dense documents and passages from the full corpus in embedding space; these candidates are ranked by a knowledge-density model and simple coverage heuristics. Second, for a portion of the retrieved content, we apply targeted transformations such as controlled rephrasing and QA synthesis to improve learnability. The resulting samples are mixed back into the training mixture to increase effective knowledge signal density. We observe consistent gains from this pipeline in ablations, while a detailed causal analysis of its benefits is left for future work.

C.2. Code Data

C.2.1. Pure-Code

We refine our internal programming dataset using a modified version of the OpenCoder filtering rules [80], introducing a calibrated relaxation to balance data quality and diversity. In our pipeline, applying OpenCoder filters generates a set of “hits” for each document, where each hit represents a violation of a heuristic rule (signaling potential noise). We categorize the corpus by these hit counts: `hit0` for clean documents (zero violations), `hit1` for one violation, and so on.

Our internal ablations reveal a clear quality-diversity trade-off: strict filtering (e.g., `hit0-only`) over-prunes the corpus, while no filtering introduces excessive noise. We find that the `hit0--6` configuration (accepting documents with up to 6 violations) yields the best overall benchmark performance, retaining a wider variety of high-signal code compared to the original strict constraints.

C.2.2. PR/Issue/Commit Data

To enhance software engineering capabilities, we construct a comprehensive dataset from GitHub repositories with over 10 stars, comprising PRs, issues, and commits. We apply strict filtering on repository popularity and content quality, and use LLMs to generate missing issue descriptions, resulting in a 5-million-sample foundation. From this, we derive four training subsets:

(1) Base PR/Issue/Commit Data: We crawl data via GHArchive and GitHub API, including full commit histories. We extract changes and validate a small portion of samples against `git diff` ground truth, then filter to 20+ mainstream languages (*e.g.*, Python, Java, C++). We strictly deduplicate against SWE-Bench Verified [13] and SWE-Bench Multilingual [14] to prevent leakage.

(2) Concatenated PR-Dialogue Data (90B tokens): We generate 90B tokens of code-editing training data by applying two Agentless-inspired templates [82]: (1) *File localization*: Given a problem description and repository structure, identify target file paths; (2) *Code repair*: Given a problem description and file content, generate precise modifications via SEARCH/REPLACE blocks.

We integrate this 90B code-editing data into two training phases with phase-specific masking strategies. In the annealing stage of pre-training, only template scaffolding is masked; in mid-training, the data is converted to chat dialogs with user prompts masked. Internal ablations show consistent gains over SWE-Bench Verified and SWE-Bench Multilingual in the cooldown stage and mid-training.

(3) Rewritten Reasoning-Oriented Data (12B tokens): From the Python subset of our base dataset, we derive bug-fix samples via LLM change-type annotation. We apply two concise rewriting strategies: (1) *Reasoning reconstruction*: an LLM reconstructs the PR author’s problem-solving process (problem analysis, root cause identification, solution design, and code implementation), injected into PR-Dialogue format. Hallucinated/inconsistent traces are filtered via rule-based and LLM verification. (2) *Active Reading notebooks*: PR/issue/commit data is converted into structured learning outlines (motivation, root causes, design decisions, insights), then synthesized into coherent technical notes. These rewritten datasets (~12B tokens) are incorporated during mid-training, yielding further gains on SWE-Bench Verified.

(4) Environment-based Seed Data. We curate executable environments derived from raw PR, issue, and commit records using the environment building pipeline described in Appendix E.2.2. Candidate samples are rigorously filtered to ensure test-patch inclusion and validated via strict rule-based criteria to guarantee environmental reproducibility. Furthermore, selected issues undergo targeted rewriting to augment data quality and coverage. The resulting dataset comprises hundreds of thousands of seed samples, including problem descriptions, code changes, and test functions, and serves as the foundational bedrock for enhancing agentic coding capabilities, driving significant performance gains in downstream agent tasks.

C.3. Mathematics & STEM Data

To enhance reasoning capabilities and elicit intelligence from knowledge, we curate a large-scale mathematics and STEM dataset. Beyond the standard Common Crawl data used in prior works [180, 181], we leverage our in-house StepCrawl system to harvest a massive scale of additional mathematics-related data. Specifically, we implement a filtering pipeline inspired by MegaMath [181], utilizing an ensemble of internal classifiers alongside FineMath [182]. This allows us to retain hundreds of billions of mathematics-related tokens distinct from Common Crawl. We further collect a diverse 100M-sample educational dataset encompassing exercises, quizzes, and instructional content. This collection bridges the gap between academic theory and professional application, covering domains from K-12 mathematics/physics/chemistry and humanities to adult vocational exams (CPA, Legal). Early-stage experiments confirm that this problem-solving data is crucial for optimizing token efficiency during pre-training.

C.4. Data Infrastructure

Our data construction and curation pipeline runs on a high-throughput in-house data infrastructure system designed for large-scale deduplication, mining, and model-inference filtering. We operate hybrid

CPU/GPU clusters with distributed frameworks such as **Spark** and **Ray** to execute both large-volume processing (e.g., minhash-based deduplication) and model-driven curation workloads (e.g., embedding generation and classifier/LM inference), backed by a storage layer spanning object storage (OSS), HDFS, and JuiceFS for efficient reads/writes of raw corpora and intermediate artifacts.

C.5. Data Ablations Setting

To rigorously assess data quality and the impact of curation strategies, we conduct an extensive ablation suite using the 30B-A3B MoE architecture trained with the Muon optimizer, consistent with the mainline settings (Table 9). Adhering to a strict token efficiency protocol, we set a fixed training budget for all experiments. Models are evaluated on the comprehensive benchmarks listed in Section 6.1, alongside a series of carefully designed held-out compression (perplexity) test sets. We observe that compression metrics often provide a more direct measure of knowledge capacity, offering signals complementary to mainstream benchmarks.

Internal experiments on the 30B-A3B MoE model demonstrate its superior performance and stability compared to smaller proxies. While smaller models are computationally cheaper, they often fail to capture the nuances of complex reasoning and lack the capacity to memorize long-tail patterns, leading to an artificial bias towards data repetition. Empirically, the 30B-A3B size offers stronger stability and better fidelity to full-scale trends.

D. Post Training Details

This section describes the post-training process that refines the base model into a high-performance agentic system, covering SFT with rigorous data processing and quality control, followed by large-scale RL to further improve reasoning, tool use, and generalization.

D.1. SFT Details

D.1.1. SFT Data Processing Pipeline

Across all domains, we apply a unified data processing pipeline that emphasizes answer verifiability, reasoning quality, and execution realism. To ensure overall data integrity, the aggregated dataset undergoes a strict two-stage filtration process:

1. **Rule-based Filtering:** We eliminate low-quality data exhibiting degenerate patterns, such as infinite repetition, harmful content, and personally identifiable information.
2. **Model-based Filtering:** We utilize specialized models to detect and filter out linguistically inconsistent data. By identifying and removing samples with unnatural language mixing, we significantly refine the dataset’s linguistic purity and overall quality.
3. **Decontamination:** We conduct comprehensive benchmark decontamination to prevent test set leakage. This involves both exact matching (with digit masking to catch numerical modifications) and N -gram matching.

This process yields a final refined dataset of 871k samples, totaling 7.23B tokens. The detailed distribution of the SFT data is presented in Table 3.

This section details the large-scale RL post-training, covering data curation, asynchronous search-agent training, and ablations on dense and MoE models.

D.2. RL Details and Ablations

D.2.1. Data Curation

We curate the RL training dataset by aggregating problems from open-source collections and competition archives spanning competitive coding, STEM, and synthetic data for general RLVR training. To prevent data contamination, we strictly exclude problems from competitions held during 2024–2026. The dataset is further augmented with: (i) synthetic arithmetic problems involving 11–13 digit integers; (ii) a generator–validator pipeline that synthesizes additional test cases for coding tasks; and (iii) synthetic environments for general reasoning tasks, such as puzzle and instruction following.

We apply a two-stage filtering process. First, deterministic rule-based pruning removes prompts containing images, external links, or open-ended requirements without a unique final answer. Second, an accuracy-based filter excludes trivial or degenerate problems. During training, each batch is constructed by sampling from different domains according to predefined sampling probabilities.

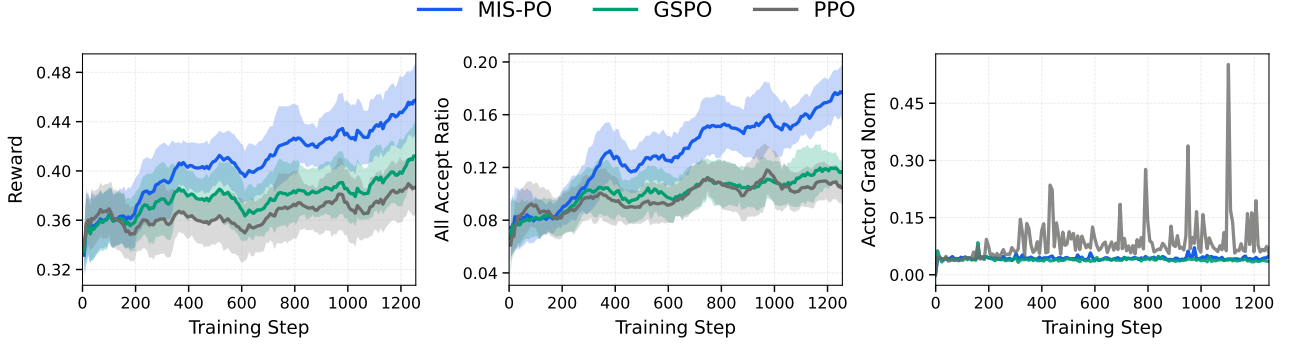
D.2.2. Reward System

Verifiable Rewards. For STEM tasks, we employ gpt-oss-120b [33] as the verifier model, using the following structured prompt (originally in Chinese) to rigorously assess final-answer correctness. For coding tasks, we utilize sandboxes to validate code execution against test cases with soft reward.

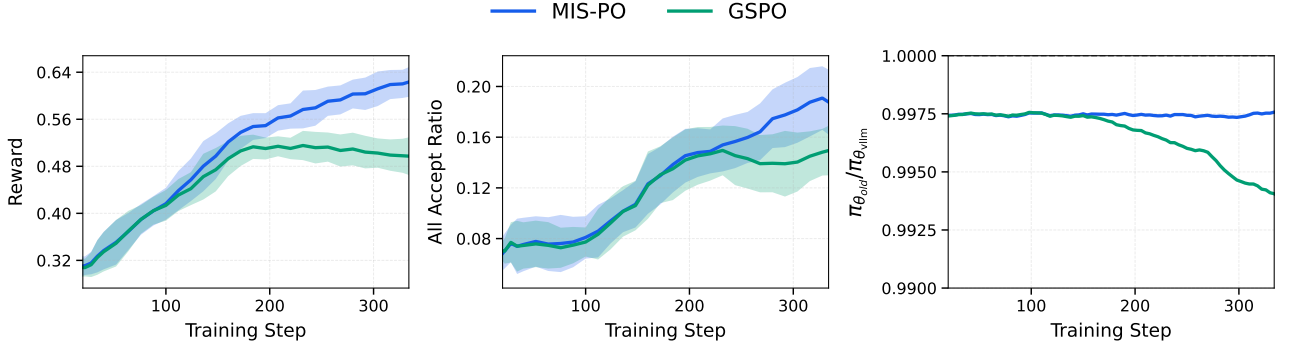
```
You are a strict grader. Below you are given the problem, the student's answer, and the reference answer. Please determine whether the student's answer is correct according to the rules below.
Grading procedure:
1. Overall check: If the student's response is incomplete, lacks a clear final answer, or contains repeated content multiple times → mark as incorrect.
2. Final-answer match: Extract the student's explicit final answer and compare it with the reference answer:
    • If they are exactly equivalent semantically or mathematically → proceed to process check.
    • If numerical computation is involved and the discrepancy is solely due to rounding → proceed to process check.
    • Otherwise → mark as incorrect.
3. Process check: Carefully verify each reasoning step:
    • If there are errors, contradictions, obvious irrelevance to the problem, or the student merely copies the prompt without a substantive solution → mark as incorrect.
    • If the solution process is correct, clear, and consistent → mark as correct.
4. Format requirements: If the problem requires a specific format (e.g., units, step-by-step answers, or explicit equations) and the student does not satisfy it → mark as incorrect.
5. Multiple sub-questions: If the problem contains multiple sub-questions, the student must answer all of them correctly to be marked correct.
6. Other cases: If the above rules do not cover the situation, make an overall judgment from the perspective of whether the student truly knows how to solve the problem.
Output requirement:
Your final output must be strictly one of the following:
    • <correct> True </correct>
    • <correct> False </correct>
Now begin:
<question>
{question}
</question>
<student_answer>
{student_answer}
</student_answer>
<reference_answer>
{reference_answer}
</reference_answer>
```

D.2.3. RL Ablation Details

MIS-PO vs. GSPO. To rigorously validate the effectiveness of our method, we benchmark MIS-PO against GSPO [36] on both Dense and MoE architectures. We select GSPO as the primary baseline because it represents a competitive strategy for reducing the gradient variance inherent in importance sampling. In our implementation, we extend the original GSPO estimator to the actor-critic setting



(a) **Comparison on the dense model.** While GSPO also effectively reduces the variance of the actor gradient norm, its efficiency is inferior to that of MIS-PO. Under the same iteration budget, MIS-PO achieves higher rewards and all acceptance ratio.



(b) **Comparison on the MoE model.** (1) **Efficiency:** MIS-PO demonstrates superior sample efficiency, achieving higher rewards with accelerated convergence, whereas GSPO plateaus around iteration 200. (2) **Stability:** GSPO exhibits an increasing training-inference discrepancy during training, quantified by the density ratio $\pi_{\theta_{\text{old}}}/\pi_{\theta_{\text{vllm}}}$ (where $\pi_{\theta_{\text{vllm}}}$ is the rollout policy in the inference backend and $\pi_{\theta_{\text{old}}}$ is the pre-update policy snapshot in the training backend). Conversely, MIS-PO consistently maintains this discrepancy within a stable range.

Figure 7: **Performance comparison between MIS-PO and GSPO.** The top figure (a) shows results on the dense model, and the bottom figure (b) shows results on the MoE model. MIS-PO consistently outperforms GSPO in both efficiency and stability across different architectures.

by integrating its Generalized Importance Sampling mechanism into the actor loss. Specifically, we replace the standard token-level importance sampling ratio with the geometric mean of trajectory-level ratios. The resulting actor loss is formulated as follows ($\gamma = \lambda = 1$):

$$r_{\tau}(\theta) = \left(\prod_{t=0}^{T-1} \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \right)^{\frac{1}{T}} \quad (14)$$

$$\hat{A}_t = \hat{R} - V_{\phi}(s_t) \quad (15)$$

$$\mathcal{L}_{\text{actor}}^{\text{GSPO}} = -\mathbb{E}_{\tau \sim \pi_{\theta_{\text{vllm}}}} \left[\mathbb{I}(x_t) \cdot \mathbb{I}(\bar{\rho}(\tau)) \cdot \min(r_{\tau}(\theta) \hat{A}_t, \text{clip}(r_{\tau}(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right] \quad (16)$$

To ensure a fair comparison, we apply the same token- and sample-level masking strategies used in MIS-PO to exclude data with significant training-inference mismatches. Regarding the clip ratio ϵ , we conduct a grid search over $\{1, 2, 3, 4\} \times 10^{-4}$. We adopt $\epsilon = 10^{-4}$ for all experiments primarily because it achieves the best benchmark performance after 200 RL training steps. Additionally, we observe that this setting yields a clip fraction of approximately 15%, consistent with the original GSPO [36].

Figure 7 presents the comparative results. Empirically, MIS-PO demonstrates superior sample efficiency and scalability compared to GSPO. Crucially, MIS-PO effectively constrains the training-inference mismatch within a stable range. This stability proves particularly critical for the large-scale RL training of MoE models, where the baseline GSPO fails to maintain consistent convergence.

Extended Training Dynamics on MoE. To further validate the scalability of our method, we conduct an extended training run of MIS-PO on the MoE model using a challenging dataset. As illustrated in Figure 8, the model maintains a continuous upward trend in rewards, stable actor gradient norms, and well-controlled entropy levels. These results empirically confirm that MIS-PO is reliability for large-scale MoE off-policy RL training.

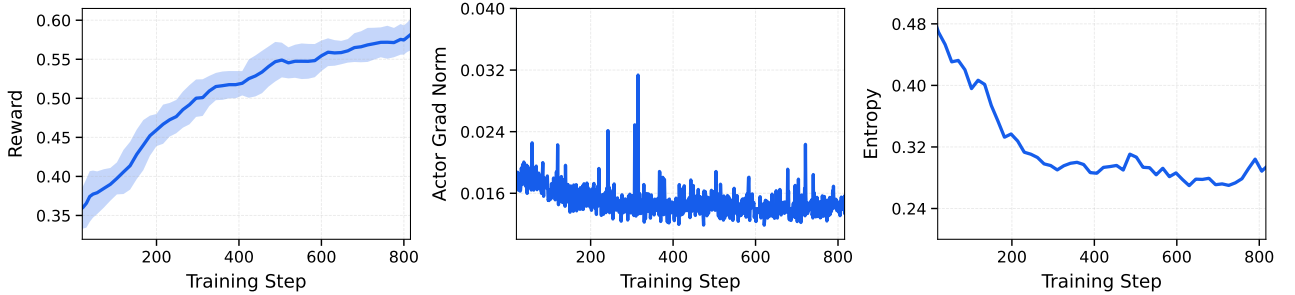


Figure 8: **Extended training dynamics of MIS-PO on the MoE model.** The metrics include Reward (left), Actor Gradient Norm (middle), and Entropy (right). Notably, the middle panel displays the raw gradient norm without smoothing or downsampling to highlight the stability of the optimization.

D.2.4. Search Agent

Regarding the training architecture, the early client-server one-step off-policy framework is severely bottlenecked by long-tail latency: approximately 5% of samples accounted for roughly 80% of the generation cost. However, our observations indicate that the policy exhibits strong robustness to staleness, maintaining stable performance even with a latency of approximately 20 steps. Consequently, we adopt the FullyAsync paradigm, decoupling generation and updates into a completely asynchronous process. Furthermore, to minimize inference overhead during multi-turn interactions, we implement sticky scheduling, where the same session is consistently dispatched to the same node to maximize KV-cache reuse. Overall, this configuration achieves an approximate 10× efficiency gain while maintaining training stability.

Throughout the training process, the FullyAsync paradigm demonstrates robust stability, evidenced by a sustained increase in rewards and a Truncated Importance Sampling (TIS) truncation rate maintained within a controllable range, thereby indicating limited policy drift induced by asynchrony. Notably, we observe that distinct from the limited scalability of “RL from zero” regarding training budgets, injecting task-relevant knowledge and tool-use priors during the mid-training phase elicited significantly higher performance gains and a more stable emergence of capabilities during the RL.

Discussion. To rigorously evaluate agentic competence isolated from parametric memorization, we focus on the *tool-usage gain*, defined as:

$$\Delta_{\text{tool}} = \text{Score}_{\text{with tools}} - \text{Score}_{\text{no tools}}$$

Model	BrowseComp	BrowseComp-ZH	GAIA	xbench DeepSearch-2505	xbench DeepSearch-2510	Avg Gain
Agent $\Delta_{\text{avg@3}}$ (Metric: Pass Rate %)						
Step 3.5 Flash*	1.5 $\blacktriangle 50.1$	25.0 $\blacktriangle 41.9$	17.0 $\blacktriangle 67.5$	26.0 $\blacktriangle 57.7$	11.3 $\blacktriangle 42.7$	52.0
Kimi K2-Thinking*	3.6 $\blacktriangle 37.9$	23.8 $\blacktriangle 38.5$	18.8 $\blacktriangle 36.6$	28.7 $\blacktriangle 39.3$	14.3 $\blacktriangle 27.0$	35.9
Kimi K2.5*	7.4 $\blacktriangle 53.2$	40.3 $\blacktriangle 22.0$	26.7 $\blacktriangle 49.2$	36.0 $\blacktriangle 40.3$	19.7 $\blacktriangle 36.6$	40.2
DeepSeek V3.2	8.1 $\blacktriangle 43.3$	41.2 $\blacktriangle 23.8$	23.4 $\blacktriangle 51.7$	35.7 $\blacktriangle 41.3$	18.7 $\blacktriangle 30.6$	38.1
GLM-4.7	3.4 $\blacktriangle 48.6$	30.2 $\blacktriangle 36.4$	19.6 $\blacktriangle 26.5$	29.7 $\blacktriangle 34.6$	19.3 $\blacktriangle 23.4$	33.9
MiniMax M2.1	1.3 $\blacktriangle 46.1$	10.1 $\blacktriangle 37.7$	15.4 $\blacktriangle 30.9$	18.7 $\blacktriangle 46.6$	6.0 $\blacktriangle 36.3$	39.5
MiMo-V2 Flash	0.9 $\blacktriangle 44.5$	12.9 $\blacktriangle 38.3$	12.9 $\blacktriangle 42.3$	19.7 $\blacktriangle 49.6$	6.3 $\blacktriangle 13.7$	37.7
Gemini 3.0 Pro	25.2 $\blacktriangle 12.6$	-	32.1 $\blacktriangle 44.5$	45.0 $\blacktriangle 32.0$	-	29.7
Claude Sonnet 4.5	1.4 $\blacktriangle 22.7$	21.2 $\blacktriangle 19.6$	16.2 $\blacktriangle 54.7$	24.7 $\blacktriangle 42.6$	7.3 $\blacktriangle 37.7$	35.5

Table 11: **Impact of Tool Usage on Agent Performance.** Each cell displays the **Baseline Score** (internal knowledge only) followed by the $\blacktriangle$ **Performance Gain** achieved by enabling search tools. The final score is the sum of both values. **Avg Gain** highlights the model’s ability to leverage external information to improve results. Models marked with * denote tool results measured under a 256K setting; the setting for other models is unspecified.

This metric decouples the model’s inherent knowledge from its ability to dynamically leverage external tools. As detailed in Table 11, **Step 3.5 Flash** demonstrates the most robust capability to leverage external information, achieving the highest average gain (52.0) and leading significantly on complex benchmarks such as GAIA and xbench-DeepSearch.

This distinction is critical because high absolute scores on benchmarks like BrowseComp can sometimes stem from strong internalized knowledge rather than effective search strategies. A smaller Δ_{tool} in a high-performing model may ambiguously indicate either high efficiency (the model already “knows” the answer) or a failure to effectively utilize tools to improve results. Conversely, a large Δ_{tool} explicitly signals the model’s proficiency in bridging knowledge gaps through retrieval. Therefore, we argue that future optimization should not merely chase higher absolute scores (“benchmark grinding”), but should aim to maximize this Δ_{tool} in long-context, evidence-critical scenarios. This ensures the agent is truly mastering the *process* of information retrieval and reasoning, rather than overfitting to static knowledge or benchmark artifacts.

D.3. Tool-integrated Reasoning and Parallel Reasoning

In this section, we introduce two primary methodologies for test-time scaling in Step 3.5 Flash: tool-integrated reasoning and parallel reasoning.

Tool-integrated Reasoning For complex reasoning tasks, we integrate the model with a Python interpreter to facilitate tool-assisted reasoning. In this framework, the model operates within a sandbox to iteratively think and execute code for computational, simulation, and visualization purposes. In our experiments, we evaluate on AIME 2025, HMMT 2025, IMO-AnswerBench, GPQA, HLE_{text}, and ARC-AGI-1 with a 100-turn limit. As shown in Table 12, tool-integrated reasoning significantly enhances performance across challenging mathematics, STEM, and puzzle benchmarks, highlighting the advanced agentic reasoning capabilities of Step 3.5 Flash.

Tool-integrated Parallel Reasoning We present a preliminary exploration of extending PaCoRe to a multi-turn interactive environment. By design, PaCoRe preserves the standard LLM message

Benchmark	Step 3.5 Flash	Step 3.5 Flash w. Python
AIME 2025	97.3	99.8 (+2.5)
HMMT 2025 Feb.	98.4	98.7 (+0.3)
HMMT 2025 Nov.	94.0	98.0 (+4.0)
IMO-AnswerBench	85.4	86.7 (+1.3)
GPQA-Diamond	83.5	84.4 (+0.9)
HLE _{text}	23.1	26.5 (+3.4)
ARC-AGI-1	54.8	56.5 (+1.7)

Table 12: Comparison of Step 3.5 Flash and Step 3.5 Flash w. Python.

interface. This compatibility allows for seamless integration into existing agentic frameworks that utilize multi-turn tool interaction. To adapt PaCoRe to this setting, we implement a state-aware input serialization protocol as shown in Table 14.

We evaluate this approach on the GPQA and HLE_{text} benchmarks using Step 3.5 Flash equipped with a Python interpreter. As shown in Table 13, extending parallel reasoning to these agentic loops yields significant performance improvements over the standard reasoning baseline. These findings demonstrate that PaCoRe effectively generalizes to environments requiring interactive feedback, highlighting a promising avenue for agentic test-time scaling.

Benchmark w. Python	Step 3.5 Flash	Step 3.5 Flash + PaCoRe
GPQA-Diamond	84.4	85.7 (+1.3)
HLE _{text}	26.5	28.2 (+1.7)

Table 13: Comparison of Step 3.5 Flash w. Python and the same model with PaCoRe test-time scaling.

E. Detailed Evaluation Protocols and Prompts

This section provides the implementation details for our evaluation suite. We outline the specific prompt templates, few-shot configurations, and the judge models employed across different benchmarks. For complex metrics, such as those used in long-context or reasoning tasks, we also detail the underlying calculation logic and scoring criteria to ensure reproducibility. In the templates provided below, {question} denotes the placeholder for the textual problem description, while other placeholders (e.g., {test}, {context}) represent task-specific information.

E.1. Evaluation Details of Pre-trained Models

E.1.1. General language understanding and reasoning benchmarks

BBH. We use the official CoT-prompts ⁷ of BBH [136], with only "Q:" and "A:" replaced by "Problem:" and "Solution:" as follows:

```
Problem:
{question}
```

⁷<https://github.com/suzgunmirac/BIG-Bench-Hard/tree/main/cot-prompts>

Panel A: Standard User Query (Last role: `user`)

You are given a problem and a list of reference responses. Your job is to analyze these references and provide your own response.

Original Problem:
`{{ original_content }}`

Reference Responses:
Note: Some references may contain `<tool_call>` tags indicating tool calls the reference intended to make. These tool calls have NOT been executed - they are shown only as reference for your analysis.
`{% for response in ref_responses %}`
Reference `{{ loop.index }}`:
`{{ response }}`
`{% endfor %}`

Now, based on the original problem and reference responses above, please provide your own comprehensive solution.

Panel B: Tool Observation (Last role: `tool`)

You are given a tool response and a list of reference responses analyzing it. Your job is to analyze these references and provide your own response.

Original Tool Response:
`{{ original_content }}`

Reference Responses:
[Same preamble regarding unexecuted tool calls as in Panel A]
`{% for response in ref_responses %}`
Reference `{{ loop.index }}`:
`{{ response }}`
`{% endfor %}`

Now, based on the original tool response and reference responses above, please provide your own comprehensive analysis and next steps.

Table 14: **Input serialization templates for Tool-integrated PaCoRe.** We introduce distinct templates to handle the initial user query (Panel A) and subsequent tool observations (Panel B). Note that `tool_calls` within reference branches are serialized as text for analysis.

Solution:

MMLU. We use the official evaluation metric of MMLU [137] with 5-shot. We employ the following task-specific system prompt:

The following are multiple choice questions (with answers) about {category}.

The corresponding question prompt is structured as follows:

{question}
Answer:

MMLU-Redux. We use the official evaluation metric of MMLU-Redux [138] with 5-shot. and employ the following question prompt:

Answer the question and place the option (A/B/C/D...) inside `\boxed{}`.
{question}

MMLU-Pro. We follow the official evaluation metric of MMLU-Pro [139] with 5-shot. All evaluations use the following system prompt:

The following are multiple choice questions (with answers) about {category}. Think step by step and then output the answer in the format of "The answer is (X)" at the end.

The question prompt is structured as follows, with a deliberate trailing space after the final period:

Question: {question}
Answer: Let's think step by step.

Notably, we observe that a subset of the original MMLU-Pro dataset (470 out of 12,102 questions) contained an inconsistent leading space before the ground-truth options. We explicitly remove these spaces to mitigate potential formatting bias and ensure evaluation consistency.

HellaSwag. We use the official evaluation metric of HellaSwag [140] with 10-shot. We employ the following question prompt:

Question: {question}
A. {option_0}
B. {option_1}
C. {option_2}
D. {option_3}
Answer:

WinoGrande. We use the official evaluation metric of WinoGrande [141] with 5-shot. The question prompt is structured to present the binary choices clearly:

Question: {question}
Options:
A. {option_0}
B. {option_1}

Answer:

GPQA. We use the official evaluation metric of GPQA [142] with 5-shot. The question prompt is structured to present the choices clearly:

Question: {question}
Options:
A. {option_0}
B. {option_1}
C. {option_2}
D. {option_3}

Answer: Let's think step by step.

SuperGPQA. We use the official evaluation metric of SuperGPQA [143] with 5-shot. The question prompt follows a Chain-of-Thought (CoT) structure, where each few-shot example includes a step-by-step derivation leading to the final answer:

```
Question:
{question}

Answer: Let's think step by step.
```

SimpleQA. We use the official evaluation metric of SimpleQA [144] with 5-shot. As SimpleQA requires open-ended short answers, we employ an LLM-based judgement for evaluation, specifically using gpt-oss-120b [33] as the judge model. The question prompt is formatted as a concise query:

```
Question: {question} Answer:
```

E.1.2. Mathematics reasoning benchmarks

GSM8K. We use the official evaluation metric of GSM8K [145] with 8-shot. The question prompt is designed to elicit CoT reasoning by using the following template:

```
Q: {question}
A: Let's think step by step.
```

MATH. We use the official evaluation metric of MATH [146] with 4-shot. The question prompt is structured with explicit problem and solution delimiters:

```
Problem:
{question}

Solution:
```

E.1.3. Coding benchmarks

HumanEval. We use the official evaluation metric of HumanEval [147] with 3-shot. The question prompt is structured with three ground-truth examples to provide contextual guidance for code generation:

```
# Below are the ground-truth solutions:

def add_two_numbers(a, b):
    """ Given two numbers a and b, return the sum of a and b. """
    # get the sum of a and b
    sum_of_a_and_b = a + b
    return sum_of_a_and_b

def reverse_list(some_list: list) -> list:
    """ Given a list, return a reversed copy of the list. """
    new_list = []
    # iterate over the list
    for item in some_list:
        # insert item into new list
        new_list.insert(0, item)
```

```

    return new_list

def fast_reverse_list(some_list: list) -> list:
    """ Given a list, return a reversed copy of the list. Be fast! """
    # use faster built-in reverse
    some_list.reverse()
    return some_list
{question}

```

MBPP. We follow the official evaluation metric of MBPP [148] with 3-shot.

HumanEval+. We follow the official evaluation metric of HumanEval+ [149] with 3-shot.

MBPP+. We use the official evaluation metric of MBPP+ [149] with zero-shot. We employ a structured instruction prompt that specifies the task requirements and includes a sample test case for alignment:

```

You are an expert Python programmer, and here is your task:
{question}
Your code should pass the test:
{test}
Here is the corresponding code:
```python

```

**MultiPL-E.** We use the official evaluation metric of MultiPL-E [150] with zero-shot. We follow the official test cases to judge the generated code.

#### *E.1.4. Chinese understanding benchmarks*

**C-Eval.** We use the official evaluation metric of C-Eval [151] and add a 5-shot setting. We employ the following system prompt:

你是一个中文人工智能助手，以下是中国关于 {category} 考试的单项选择题，请选出其中的正确答案。

The corresponding question prompt is structured as follows:

```

{question}
答案：

```

**CMMLU.** We use the official evaluation metric of CMMLU [152] and add a 5-shot setting. We employ the following system prompt:

你是一个中文人工智能助手，以下是中国关于 {category} 考试的单项选择题，请选出其中的正确答案。

The corresponding question prompt is structured as follows:



---

```
{question}
答案:
```

---

**C-SimpleQA.** We use the official evaluation metric and LLM-based judgement protocols of Chinese SimpleQA [153]. We add a 5-shot setting and use gpt-oss-120b [33] as the judge model. We employ the following question prompt:

---

```
问题: {question}
答案:
```

---

## E.2. Evaluation Details of Post-Trained Models

In this section, we detail the evaluation protocols used to assess the post-trained models across a diverse set of agentic tasks. Our evaluations span both code-centric and general-purpose agent settings, covering software engineering, terminal interaction, deep search, research workflows, and real-world tool use. We report standardized metrics under carefully controlled environments and inference budgets to ensure fair, stable comparisons across benchmarks.

### E.2.1. Reasoning benchmarks

**CF-Div2-Stepfun.** Recent studies and advanced benchmarks emphasize the critical need to evaluate models on fresh, competition-level problem [183, 184]. We evaluate the competitive programming capabilities of our model using a custom CodeForces Div. 2 Benchmark <sup>8</sup>. The benchmark comprises 53 problems sourced from official CodeForces Div.2 contests held between September 2024 and February 2025. We develop an offline evaluation framework that utilizes a local grading mechanism as an alternative to real-time online submissions. We try to construct test cases similar to the original test cases. Specifically, we first generate enough small-scale test cases for evaluation correctness coverage, then add randomized data for large-scale testing. Finally, we performed adversarial construction of edge cases by analyzing common error patterns and "hacked" submissions from actual users. Some edge cases are also auto-generated by the stress testing technique, which keeps generating countless test cases until one can distinguish failed submissions from correct submissions. To validate the reliability of this benchmark, we run both correct and representative failed submissions selected from the original contests. Our evaluator correctly identifies 100% of the accepted submissions as "Passed", while 92.45% of the failed submissions are accurately flagged.

---

<sup>8</sup><https://huggingface.co/datasets/stepfun-ai/CF-Div2-Stepfun>

Model	Accuracy (avg@8)			Codeforces C++
	C++	Python	Java	pass@8 Rating
Step 3.5 Flash	<b>86.1%</b>	<b>81.5%</b>	77.1%	<b>2489</b>
Deepseek V3.2	81.6%	66.5%	80.7%	2319
GLM-4.7	74.1%	63.0%	70.5%	2156
Kimi K2-Thinking	67.9%	60.4%	58.5%	1976
Minimax-M2.1	59.0%	46.4%	58.0%	1869
Mimo-V2 Flash	46.9%	43.6%	39.6%	1658
Gemini 3.0 Pro	83.5%	74.1%	<b>81.6%</b>	2397
Claude Opus 4.5	72.2%	68.4%	68.9%	2100

Table 15: Full evaluation results of variable models in CF-Div2-Stepfun.

We sample 8 responses for each problem and report the average accuracy. The user prompt utilized for this process is:

```
You are a coding expert. Given a competition-level coding problem, you need to write a
{LANGUAGE} program to solve it. You may start by outlining your thought process. In the
end, please provide the complete code in a code block enclosed with `````.
{question}
```

The compilation and execution commands for C++, Python, Java are given below:

```
g++ -std=c++20 -fno-asm -fsanitize=bounds -fno-sanitize-recover=bounds --static -O2
-DONLINE_JUDGE -o code.exe code.cpp
./code.exe
```

```
python3 code.py
```

```
javac -J-Xmx544m {JAVA_CLASS_NAME}.java
java -XX:+UseSerialGC -Xmx544m -Xss64m -DONLINE_JUDGE {JAVA_CLASS_NAME}
```

To maintain consistency with competitive programming norms and avoid the inconsistent overhead associated with JIT "warm-up" periods, we use the standard Python interpreter with a double time limit rather than PyPy<sup>9</sup>. We apply this same double time limit to all Java submissions.

While the Table 15 reports raw accuracy, we recognize that problem difficulty varies significantly. Therefore, rating scores provide more robust metrics. Although frameworks like CodeELO [185] can calculate competitive ratings, current top-tier models perform so effectively in Division 2 contests that their ratings may result in statistical outliers. Furthermore, we adopt a simplified rating calculation that disregards submission time penalties by assuming all solutions are submitted at the onset of the contest. While this approach deviates from empirical competitive scenarios and may result in ratings that are not directly comparable to human participants, it provides a standardized benchmark for consistent cross-model comparison.

**LiveCodeBench-v6.** We use the official evaluation method of LiveCodeBench [9]. We employ the following system prompt:

<sup>9</sup><https://pypy.org/>

---

You are an expert Python programmer. You will be given a question (problem specification) and will generate a correct Python program that matches the specification and passes all tests.

---

The corresponding question prompt is structured as follows:

```
Question:
{question}
Format: You will use the following starter code to write the solution to the problem
and enclose your code within delimiters.
``` python
{starter_code}
```
Answer: (use the provided format with backticks)
```

---

**AIME 2025.** We use the official evaluation method of AIME 2025 [170] with repeat@64. We employ the following question prompt:

---

```
Answer the question and place the answer inside \boxed {} with MathTeX format.
{question}
```

---

**HMMT 2025 Feb./Nov.** We use the official evaluation method of HMMT 2025 [8] with repeat@64. We employ the following question prompt:

---

```
Answer the question and place the answer inside \boxed {} with MathTeX format.
{question}
```

---

**IMO-AnswerBench.** We use the official evaluation method of IMO-AnswerBench [91] with repeat@64. We employ the following question prompt:

---

```
Answer the question and place the answer inside \boxed {} with MathTeX format.
{question}
```

---

**MMLU-Pro.** We use the official evaluation method of MMLU-Pro [139]. The processing of dataset remains consistent with our pre-training MMLU-Pro evaluation methodology (see Appendix E.1.1 for details).

---

```
Answer the question and place the option (A/B/C/D...) inside \boxed{}.
{question}
```

---

**GPQA-Diamond.** We use the official evaluation method of GPQA-Diamond [142]. We employ the following question prompt:

---

```
Answer the question and place the option (A/B/C/D...) inside \boxed{
{question}}
```

---

**HLE<sub>text</sub>.** We use the official evaluation metric and LLM-based judgement protocols of HLE. We use gpt-oss-120b [33] as the judge model.

### *E.2.2. Code Agent benchmarks*

**SWE-Bench.** SWE-Bench Verified [13] is a high-quality subset of the original SWE-bench dataset, consisting of 500 software engineering tasks rigorously validated by human expert developers to ensure reliable and accurate evaluation. SWE-Bench Multilingual extends the original benchmark to a diverse set of 300 real-world software engineering tasks across 9 programming languages.

We test the software engineering agent ability of Step 3.5 Flash on SWE-Bench Verified and SWE-Bench Multilingual using our internal agent infrastructure, which is built upon the described session-router architecture. For each evaluation instance, we provision a containerized session orchestrated via Kubernetes. We then perform environment initialization specific to SWE-Bench, which includes removing future commits to prevent data leakage, as well as configuring network proxies and critical system settings. Regarding the agent scaffold, we adopted the OpenHands [131] CodeAct Agent framework, which is widely used in the research community. We enabled a default suite of four tools: `execute_bash`, `str_replace_editor`, `finish`, and `think`. The max interactive turns is set to 350.

Given the resource-intensive nature of compiled languages, we allocate 12GB of memory for the multilingual setting, whereas the verified instances are restricted to a 4GB limit. In evaluations, the tool execution timeout is set to 1200s, and the model inference parameters are: `temperature=1`, `top-p=0.95`. Following the above settings, Step 3.5 Flash reach 74.4% on SWE-Bench Verified, and 67.4% on SWE-Bench Multilingual benchmark with an average score of 4 repeat of runnings. We also cross-evaluate Step 3.5 Flash on other popular agent scaffolds: SWE-Agent [132] with the original agent pipeline settings achieving 74.2% accuracy on SWE-Bench Verified, and standard Claude Code <sup>10</sup> environment scoring 72.0% with an extended time limit of 4 hours for each instance and no time limit for single tool execution.

**Terminal-Bench 2.0.** We test the Terminal-Bench benchmark [16] within remote task-independent containers. We limit the container memory to 16GB. We have deployed an internal Artifactory repository and update the default package sources for all Docker containers. During session creation and dependency installation of the testing phases, the system will retry multiple times if an error occurs. To streamline the system-agent interaction, we modify the Terminus 2 framework so that it automatically interrupts timed-out commands and prevents subsequent commands in the same round from executing, returning a timeout warning to the agent. Accordingly, we modify the command duration control part of the original system prompt:

---

```
Keystroke duration sets the command hard timeout. The system automatically interrupts
timed-out commands and prevents subsequent commands in the same round from executing.
You can simply continue with your next round - no special action is required.
```

---

During inference, we cap the model’s single-turn output at 64k and the maximum context window at 256k for all interactions. The thinking process will be preserved in the multi-round history. If the model

---

<sup>10</sup><https://github.com/anthropics/claude-code>

output exceeds the 256k context window limit, we execute a pruning context management: Keep the problem statement and the last 50% of history before retrying. We use the inference parameters of top-p=0.95 and temperature=1. The interaction protocol is primarily conducted using XML-formatted structured responses. The agent is limited to 200 interaction rounds and will proceed directly to the testing phase once this limit is reached. The total time limit for interaction and testing is 6 hours.

To ensure consistency, we verify and refine each task’s checker against its problem statement <sup>11</sup>, which improved overall accuracy by approximately 1.5%. Each task is executed across 8 trials. Notably, 88.6% of successful trajectories are completed within 30 interactions. The final pass@8 stands at 67/89, with an avg@8 of 50.98%. Our agent achieves a 100% success rate across all 8 trials in 23 out of 89 tasks. In the successful trajectories, 9.41% of the runs triggered history pruning to manage context limits.

| Setting               | Max Output | Max Round | Timeout | Context Management | Avg@8         |
|-----------------------|------------|-----------|---------|--------------------|---------------|
| Baseline              | 64k        | 200       | 6h      | ✓                  | <b>50.98%</b> |
| Limit 16k             | 16k        | 200       | 6h      | ✓                  | 48.03%        |
| Limit 16k w/o Pruning | 16k        | 200       | 6h      | ×                  | 45.22%        |
| Rounds 100            | 64k        | 100       | 6h      | ✓                  | 50.42%        |
| Timeout 2h            | 64k        | 200       | 2h      | ✓                  | 49.72%        |

Table 16: Ablation study of inference constraints on Terminal-Bench 2.0.

The ablation study shows that Limit 16k causes the largest performance drop because the model’s long reasoning for complex tasks often exhausts the token limit before it can output the terminal commands. The further decline to 45.22% when disabling context management under the 16k limit. Meanwhile, Rounds 100 has minimal impact as most tasks finish early. The Timeout 2h decrease reflects that certain tasks involving model training, heavy compilation, or complex environment configuration require more time to complete.

### E.2.3. General Agent benchmarks

**Deep Search.** We evaluate our agent’s deep search capabilities on multiple benchmarks (e.g, **BrowseComp** [17], **BrowseComp-ZH** [18], **GAIA** [19], **xbench-DeepSearch** [20]). The results reported in Table 5 are based on the avg@3 metric; GPT-5.2 xHigh uses avg@1. The agent is equipped with a core toolset including:

- **search:** Executes multiple search queries in parallel.
- **visit:** Analyzes the content of the webpage to answer specific questions based on LLM.
- **google\_scholar:** Search for academic articles and technical literature.
- **python\_interpreter:** Runs Python code for calculations and data analysis.
- **file:** Downloads and saves files from direct URLs.

During inference, we employ a 256k-token context window with no limit on the maximum generation length. Inference is conducted with top-p = 0.95, temperature = 1.0, and presence penalty = 1.1, allowing for an execution budget of up to 400 steps.

The detailed system prompts for the agent and the LLM judge are consistent with the configurations provided in the GitHub repository associated with [98].

<sup>11</sup><https://huggingface.co/datasets/zai-org/terminal-bench-2-verified>

**BrowseComp (w. Ctx Manage).** The BrowseComp (w. Ctx Manage) result of 69.0 reported in Table 5 corresponds to the discard-all methodology evaluated on the full BrowseComp dataset. This approach, same as DeepSeek V3.2 [1], is triggered when the context length exceeds predefined thresholds, at which point the agent discards its entire context and reinitializes the operational loop. Under a maximum iteration constraint of 1000 steps, this strategy employs a context length threshold of 72k tokens for BrowseComp and 41k tokens for BrowseComp-ZH.

We also evaluate various context management strategies on a subset of 200 instances from BrowseComp, including Summary, Keep-first&lastK, Discard-all, and Multi-agent orchestration. As shown in Table 17, our model demonstrates robust adaptability across these diverse paradigms. Among single-agent strategies, Discard-all yields a competitive 66.0% accuracy. We posit that Discard-all functions as a test-time pass@k strategy, forcing the model to re-reason from scratch until a self-verified path is found. The performance follows a clear hierarchy: Multi-agent ranks highest by leveraging a master agent to decompose tasks and dispatch specialized agents for parallel reasoning, followed by Discard-all, Keep-first&lastK and Summary —closely aligns with the increase in real steps. This alignment reflects a direct trade-off between inference cost (number of steps) and accuracy, suggesting that intensive context management effectively converts increased computation into superior performance.

| Method             | Accuracy (%) | Real Steps |
|--------------------|--------------|------------|
| Step 3.5 Flash     | 49.5         | 86         |
| + Summary          | 57.0         | 131        |
| + Keep-first&lastK | 58.0         | 244        |
| + Discard-all      | 66.0         | 302        |
| + Multi-Agent      | 68.5         | 721        |

Table 17: Evaluation results of context manager methods.

**ResearchRubrics.** To evaluate deep research capabilities, we utilize the RESEARCHRUBRICS [21] benchmark. This dataset comprises 101 domain-diverse research tasks, each accompanied by 20–43 expert-written, fine-grained scoring criteria that assess factual accuracy, reasoning soundness, and clarity. We benchmark performance against two representative system families: commercial agent systems and ReAct agents.

For commercial agents, we collect reports via their official web interfaces (captured Dec 2–15, 2025) under default configurations. As shown in Table 18, the leading commercial system (Gemini DeepResearch) achieves an aggregated score of 63.69.

| Agent System        | Score |
|---------------------|-------|
| Gemini DeepResearch | 63.69 |
| OpenAI DeepResearch | 60.67 |
| Kimi Researcher     | 53.67 |
| MiniMax Agent Pro   | 51.85 |
| Qwen DeepResearch   | 49.24 |

Table 18: Performance of Commercial Agent Systems on the RESEARCHRUBRICS benchmark.

For ReAct agents, detailed performance comparisons are presented in Table 5. Our model achieves a score of **65.3**, surpassing the complex, proprietary commercial baselines. Notably, when evaluating

Gemini 3.0 Pro within our standardized ReAct framework, we observe a score of 50.1. We attribute this performance gap to insufficient search depth when addressing open-ended research questions; the model tends to rely on internal parametric knowledge rather than perform extensive external retrieval. Consequently, the generated reports lack comprehensiveness, failing to adequately cover the user’s implicit criteria.

We standardize the execution environment for ReAct agents with a maximum of 30 reasoning turns and a per-turn output limit of 16k tokens. For inference parameters, other API-based models use their default settings, and our model is configured with a temperature of 1 and top-p=0.95. All outputs are subsequently appraised by an LLM judge using a ternary grading for each criterion. To support the end-to-end research workflow, our ReAct framework provides access to the following suite of tools:

- **batch\_web\_surfer**: For concurrent web searching and multi-page browsing.
- **file**: For robust file operations, including reading, writing, and iterative editing.
- **file\_parser**: For converting files into Markdown format.
- **shell**: For interactive command execution and environment interaction.
- **todo**: For dynamic task state management and tracking.
- **tmux**: For simulating a multiplexed terminal environment with persistent sessions and scrollbar history.

**$\tau^2$ -Bench.**  $\tau^2$ -Bench [15] is an agentic benchmark that evaluates general tool-use capability in three customer service domains: airline, retail, telecom. We evaluate Step 3.5 Flash using the official settings in the original codebase. Specifically, we use the default LLM agent framework and set the temperature to 1.0, top-p to 0.95, max sequence length to 256K. The user model is set to GPT-4.1 with 0.0 temperature to ensure a stable interaction during evaluation. For the airline domain, since it has incorrect ground truth answers, we use the fixed version from Claude Opus 4.5 to ensure evaluation reliability<sup>12</sup>. For the retail and telecom domains, we also follow Claude Opus 4.5 to include a general prompt addendum to the user prompt to avoid failure modes from the user ending the interaction incorrectly<sup>13</sup>. We report an average score of 8 runs to ensure stable evaluation results.

#### *E.2.4. General benchmarks*

**Arena-Hard-v2.0.** We use the official evaluation metric of Arena-Hard-v2.0 [155] and use GPT-4.1 [186] as the judge model.

**MultiChallenge.** We use the official evaluation metric of MultiChallenge [157] with o3-mini [187] as the judge model. This follows findings from the GPT-5 [188] release that GPT-4o [186] frequently mis-scores complex responses, leading to underestimated results.

**IFBench.** We use the official evaluation method of IFBench [156].

#### *E.2.5. Long Context benchmarks*

**LongBench v2.** We use the official evaluation method of LongBench v2 [158].

<sup>12</sup><https://github.com/sierra-research/tau2-bench/pulls/chrisgorgo>

<sup>13</sup><https://github.com/anthropics/model-cards/tree/main/claude-opus-4-5-20251101/tau2>

**MRCR-8needle.** For MRCR-8needle [159] benchmark, we report the Area Under Curve (AUC) metric, following the protocol established by ContextArena<sup>14</sup>. Specifically, we use the **AUC@128k** metric, which provides a single holistic score summarizing performance across context lengths up to 131,072 tokens.

The AUC is calculated by plotting the average retrieval accuracy for each context bin (ranging from 8k to 128k) against the bin’s maximum context length. We apply the trapezoidal rule on a linear scale to measure the area under the resulting curve, which is then normalized by the total context width (128k minus the initial bin size) to yield a percentage score between 0% and 100%. This metric effectively penalizes performance degradation as difficulty increases with longer context sequences.

**FRAMES-Oracle.** We use the official evaluation metric of FRAMES [160]. Since our focus is on long-context capabilities, we specifically report results for the **Oracle Prompt** subset. In this setting, the model is provided with the question alongside all ground-truth Wikipedia articles used during human annotation. This configuration serves as an upper bound for model performance, simulating a perfect retrieval system that delivers all relevant context to the model.

**RepoQA.** We use the official evaluation method of REPOQA [161].

### E.3. Internal Evaluation - Benchmarks and Methodology

#### E.3.1. Data Analysis Benchmark

To reliably assess Step 3.5 Flash ’s ability to perform practical data-analysis tasks in the Claude Code environment, we develop an internal Data Analysis Benchmark for evaluating end-to-end analytical problem solving under realistic business constraints. The benchmark is constructed by systematically distilling senior practitioners’ tacit expertise into a rubric-grounded evaluation suite. This approach captures the ambiguity and contextual nuance of real-world analytics while ensuring consistent evaluation through standardized rubrics and verifiable ground-truth artifacts.

The benchmark is constructed using an expert-driven, rubric-based protocol to ensure domain authenticity and scoring reliability. Ten senior data analytics leaders from major Chinese internet companies, each with over 15 years of experience, contributed real-world business cases through structured interviews that elicited core analytical patterns and decision logic. This process yields representative tasks paired with expert-endorsed solution strategies.

Interview materials are normalized into machine-consumable tasks, each comprising a problem statement, a CSV dataset, a reference analysis, and a weighted checklist-style scoring rubric. The resulting benchmark contains 50 items spanning diverse analytical intents, with an average of 26.9 rubric items per task. Quality is ensured through iterative expert review, aligning task definitions, data, reference solutions, and evaluation criteria to improve validity and reproducibility.

We further implement a unified end-to-end evaluation framework covering task execution, automated scoring, and report synthesis. The framework supports code-based, research-oriented, and text-based analyses within a single pipeline, enabling scalable and reproducible evaluation across heterogeneous environments with low integration overhead.

**Evaluation Method.** Each task is evaluated by a model-based evaluator that scores generated outputs against expert-defined rubrics, with results averaged over 3 identical runs to reduce stochastic

---

<sup>14</sup><https://contextarena.ai/>



variance and ensure reliable, comparable cross-model evaluation.

| Model           | Avg@3(%) |
|-----------------|----------|
| Claude Opus 4.5 | 45.0     |
| Step 3.5 Flash  | 39.6     |
| GPT-5.2         | 39.3     |
| Gemini 3.0 Pro  | 33.6     |
| Deepseek V3.2   | 27.9     |

Table 19: Evaluation Results on the Data Analysis Benchmark

**Evaluation Results.** Table 19 presents the results on the Data Analysis Benchmark. Claude Opus 4.5 ranks first overall, while Step 3.5 Flash achieves a strong second place (39.58%) and remains very close to GPT-5.2 (39.31%). Its competitive performance may be partly related to relatively good adaptation to the Claude Code environment. In addition, Step 3.5 Flash demonstrates a favorable speed-capability trade-off, maintaining solid analytical quality while delivering faster responses. The results position Step 3.5 Flash as a highly efficient and competitive option for real-world data analysis tasks.

### *E.3.2. Consulting and Recommendations Benchmark*

To rigorously evaluate Step 3.5 Flash in real-world advisory scenarios, we curate a benchmark of 500 diverse queries sourced from authentic social platforms such as Reddit, Stack Exchange, and various community forums. These queries represent authentic user intent across everyday life, academic learning, entertainment, and professional workplace contexts.

Here, we implement an "Anchor-Based" scoring framework to evaluate candidate models. In this process, we first utilize leading models, including GPT-5.2, Claude Opus 4.5, and DeepSeek V3.2 to generate independent responses for each query. These high-level outputs are then synthesized and refined by human experts to create a Reference Response as Ground Truth. This reference serves as a high-quality "Anchor" with a standardized performance value of 88/100.

We then measure the performance of the models across four critical dimensions, applying a rigorous scoring rubric, including Usefulness, Logic, Instruction Following, and Tone. Usefulness assesses whether the model delivers a ready-to-use solution that meaningfully resolves the task with expert-level depth, actionable steps, and feasible recommendations. Logic evaluates factual accuracy and structural soundness, checking for hallucinations, incorrect citations, invalid conclusions, or causal and temporal inconsistencies, as well as overall coherence and argument flow. Instruction Following measures adherence to both explicit constraints (*e.g.*, formatting, length, and stated requirements) and implicit contextual expectations embedded in the user query. Tone assesses communicative quality, including appropriateness of language and register, clarity in unpacking complex reasoning, and calibrated expression that avoids overconfidence while clearly signaling uncertainty when appropriate.

We employ a Hybrid LLM-as-a-Judge system. Recognizing that different frontier models have distinct evaluative strengths, we assign specific scoring responsibilities as follows: Logic, Instruction Following, and Usefulness: These three dimensions are evaluated by GPT-5.2, leveraging its industry-leading capabilities in factual verification, constraint checking, and objective problem-solving. Tone: This dimension is evaluated by Claude Opus 4.5, utilizing its superior nuance in linguistic style, emotional calibration, and "human-like" resonance. Judge reliability is validated through an alignment study with human experts, yielding a high Pearson correlation between AI- and human-assigned scores. Final

scores are computed using equal weighting across the four dimensions (25% each), ensuring a balanced assessment that jointly reflects technical correctness and communicative quality.

| Model           | Average | Usefulness | Logic | Tone  | Instruction-following |
|-----------------|---------|------------|-------|-------|-----------------------|
| GPT-5.2         | 77.8%   | 77.2%      | 81.9% | 73.0% | 79.6%                 |
| Kimi K2.5       | 72.2%   | 77.1%      | 62.1% | 72.7% | 77.3%                 |
| Gemini 3.0 Pro  | 70.6%   | 73.9%      | 61.7% | 72.3% | 74.4%                 |
| Step 3.5 Flash  | 70.5%   | 73.3%      | 62.1% | 72.4% | 74.2%                 |
| Deepseek V3.2   | 70.3%   | 72.5%      | 64.4% | 71.2% | 72.9%                 |
| GLM-4.7         | 70.3%   | 73.5%      | 61.5% | 72.5% | 73.6%                 |
| Claude Opus 4.5 | 68.5%   | 69.7%      | 66.5% | 65.9% | 72.1%                 |
| Mimo-V2 Flash   | 67.9%   | 71.5%      | 58.0% | 70.6% | 71.4%                 |
| Minimax M2.1    | 67.1%   | 70.7%      | 60.1% | 67.2% | 70.4%                 |

Table 20: Evaluation results on the Consulting and Recommendations Benchmark

**Evaluation Results.** Table 20 shows that Step 3.5 Flash achieves an average Score of 70.5% on the Consulting and Recommendations Benchmark, securing the 4th position overall. Step 3.5 Flash matches Gemini 3.0 Pro performance across all dimensions, achieving comparable Pro-level scores (70.5% vs. 70.6%) while offering substantially lower inference cost and latency. Unlike many fast models that trade speed for degraded reasoning quality, Step 3.5 Flash surpasses larger models in the Logic dimension, reducing hallucinations and logical failures and making it well suited for automated consulting workflows where factual integrity is critical.

### *E.3.3. Step 3.5 Flash + Step-GUI*

To validate Step 3.5 Flash’s efficacy in real-world agentic scenarios, we evaluate on **AndroidDaily Hard** [189], a challenging benchmark designed for Chinese mobile application environments. This benchmark comprises compositional tasks spanning e-commerce transactions, multimedia interactions, and daily mobile operations, offering a naturalistic testbed for assessing GUI agent capabilities in complex, multi-step workflows representative of production deployments.

We empirically investigate two architectural instantiations: (1) **Step-GUI** [189], a lightweight on-device agent (Edge Only) that executes tasks autonomously using local computational resources, and (2) **Step 3.5 Flash + Step-GUI**, an edge-cloud collaborative framework wherein Step 3.5 Flash functions as a cloud-based reasoning orchestrator that synthesizes high-level task plans, decomposes them into executable primitives via the GUI-MCP protocol, and delegates low-level control to the on-device Step-GUI agent. This hierarchical architecture exploits the complementary strengths of cloud-scale reasoning and edge efficiency: Step 3.5 Flash’s 11B active parameters enable sophisticated multi-step planning and contextual understanding, while Step-GUI ensures low-latency action execution and privacy-preserving local control.

**Quantitative Results.** The edge-cloud collaborative paradigm achieves a success rate of **57.0%** on AndroidDaily Hard, substantially outperforming the edge-only baseline (**40.0%**). This result suggests that combining strong cloud-side reasoning with efficient edge execution is an effective strategy for navigating deployment constraints in multi-round agent interactions.

**Architectural Generalization.** Critically, this collaborative pattern extends beyond mobile ecosystems to heterogeneous platforms including desktop computers and automotive infotainment systems. By decoupling cognitive orchestration (cloud) from embodied execution (edge), the framework establishes a scalable paradigm for deploying sophisticated agents in resource-constrained industrial environments—directly aligned with Step 3.5 Flash’s design objective of redefining the efficiency frontier for production-grade agentic systems. The results underscore that effective real-world agents require not only advanced reasoning capabilities but also architectures that harmonize computational distribution across infrastructure tiers.

## References

- [1] DeepSeek-AI. Deepseek-v3.2-exp: Boosting long-context efficiency with deepseek sparse attention, 2025.
- [2] Aohan Zeng, Xin Lv, Qinkai Zheng, Zhenyu Hou, Bin Chen, Chengxing Xie, Cunxiang Wang, Da Yin, Hao Zeng, Jiajie Zhang, et al. Glm-4.5: Agentic, reasoning, and coding (arc) foundation models. *arXiv preprint arXiv:2508.06471*, 2025.
- [3] LLM-Core Xiaomi. Mimo-v2-flash technical report, 2026.
- [4] Meituan LongCat Team, Bei Li, Bingye Lei, Bo Wang, Bolin Rong, Chao Wang, Chao Zhang, Chen Gao, Chen Zhang, Cheng Sun, et al. Longcat-flash technical report. *arXiv preprint arXiv:2509.01322*, 2025.
- [5] Kimi Team, Yifan Bai, Yiping Bao, Guanduo Chen, Jiahao Chen, Ningxin Chen, Ruijue Chen, Yanru Chen, Yuankun Chen, Yutian Chen, et al. Kimi k2: Open agentic intelligence. *arXiv preprint arXiv:2507.20534*, 2025.
- [6] MiniMax Team. Minimax-m2.1, 2025.
- [7] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are rnns: fast autoregressive transformers with linear attention. In *Proceedings of the 37th International Conference on Machine Learning, ICML’20*. JMLR.org, 2020.
- [8] HMMT. Hmmt 2025 feb., 2025.
- [9] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.
- [10] OpenAI. Gpt-5.2, 2025.
- [11] Google DeepMind. Gemini 3 promodel card, 2025.
- [12] Anthropic. System card: Claude opus 4.5, 2025.
- [13] OpenAI. Introducing SWE-bench verified we’re releasing a human-validated subset of swe-bench that more, 2024.
- [14] John Yang, Kilian Lieret, Carlos E. Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. Swe-smith: Scaling data for software engineering agents, 2025.
- [15] Sierra Research. tau2-bench. <https://github.com/sierra-research/tau2-bench>, 2025.
- [16] Mike A Merrill, Alexander G Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E Kelly Buchanan, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *arXiv preprint arXiv:2601.11868*, 2026.

- [17] Jason Wei, Zhiqing Sun, Spencer Papay, Scott McKinney, Jeffrey Han, Isa Fulford, Hyung Won Chung, Alex Tachard Passos, William Fedus, and Amelia Glaese. Browsecomp: A simple yet challenging benchmark for browsing agents, 2025.
- [18] Peilin Zhou, Bruce Leon, Xiang Ying, Can Zhang, Yifan Shao, Qichen Ye, Dading Chong, Zhiling Jin, Chenxuan Xie, Meng Cao, Yuxin Gu, Sixin Hong, Jing Ren, Jian Chen, Chao Liu, and Yining Hua. Browsecomp-zh: Benchmarking web browsing ability of large language models in chinese, 2025.
- [19] Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. Gaia: a benchmark for general ai assistants, 2023.
- [20] Kaiyuan Chen, Yixin Ren, Yang Liu, Xiaobo Hu, Haotong Tian, Tianbao Xie, Fangfu Liu, Haoye Zhang, Hongzhang Liu, Yuan Gong, et al. xbench: Tracking agents productivity scaling with profession-aligned real-world evaluations. *arXiv preprint arXiv:2506.13651*, 2025.
- [21] Manasi Sharma, Chen Bo Calvin Zhang, et al. Researchrubrics: A benchmark of prompts and rubrics for evaluating deep research agents. *arXiv preprint arXiv:2511.07685*, 2025.
- [22] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *arXiv preprint arXiv:2101.03961*, 2021.
- [23] Barret Zoph, Irwan Bello, Sameer Kumar, Nan Du, Yanping Huang, Jeff Dean, Noam Shazeer, and William Fedus. St-moe: Designing stable and transferable sparse expert models, 2022.
- [24] Nan Du, Yanping Huang, Andrew M Dai, Simon Tong, Dmitry Lepikhin, Yuanzhong Xu, Maxim Krikun, Yanqi Zhou, Adams Wei Yu, Orhan Firat, Barret Zoph, Liam Fedus, Maarten P Bosma, Zongwei Zhou, Tao Wang, Emma Wang, Kellie Webster, Marie Pellat, Kevin Robinson, Kathleen Meier-Hellstern, Toju Duke, Lucas Dixon, Kun Zhang, Quoc Le, Yonghui Wu, Zhifeng Chen, and Claire Cui. GLaM: Efficient scaling of language models with mixture-of-experts. In Kamalika Chaudhuri, Stefanie Jegelka, Le Song, Csaba Szepesvari, Gang Niu, and Sivan Sabato, editors, *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 5547–5569. PMLR, 17–23 Jul 2022.
- [25] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. Gshard: Scaling giant models with conditional computation and automatic sharding, 2020.
- [26] Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, Zhenda Xie, Y. K. Li, Panpan Huang, Fuli Luo, Chong Ruan, Zhifang Sui, and Wenfeng Liang. Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models, 2024.
- [27] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *CoRR*, abs/1904.10509, 2019.
- [28] Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and Gabriel Synnaeve. Better & faster large language models via multi-token prediction. *arXiv preprint arXiv:2404.19737*, 2024.
- [29] DeepSeek-AI. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- [30] LLM Xiaomi, Bingquan Xia, Bowen Shen, Dawei Zhu, Di Zhang, Gang Wang, Hailin Zhang, Huaqiu Liu, Jiebao Xiao, Jinhao Dong, et al. Mimo: Unlocking the reasoning potential of language model—from pretraining to posttraining. *arXiv preprint arXiv:2505.07608*, 2025.
- [31] Zihan Qiu, Zekun Wang, Bo Zheng, Zeyu Huang, Kaiyue Wen, Songlin Yang, Rui Men, Le Yu, Fei Huang, Suozhi Huang, Dayiheng Liu, Jingren Zhou, and Junyang Lin. Gated attention for large language models: Non-linearity, sparsity, and attention-sink-free, 2025.
- [32] StepFun Team. Step3: Cost-effective multimodal intelligence.
- [33] OpenAI. Gpt-oss-120b & gpt-oss-20b model card, 2025.

- [34] Keller Jordan, Yuchen Jin, Vlado Boza, Jiacheng You, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks, 2024.
- [35] Jason Wei, Nguyen Karina, Hyung Won Chung, Yunxin Joy Jiao, Spencer Papay, Amelia Glaese, John Schulman, and William Fedus. Measuring short-form factuality in large language models. *arXiv preprint arXiv:2411.04368*, 2024.
- [36] Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, et al. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*, 2025.
- [37] Feng Yao, Liyuan Liu, Dinghuai Zhang, Chengyu Dong, Jingbo Shang, and Jianfeng Gao. Your efficient rl framework secretly brings you off-policy rl training, August 2025.
- [38] Wenhan Ma, Hailin Zhang, Liang Zhao, Yifan Song, Yudong Wang, Zhifang Sui, and Fuli Luo. Stabilizing moe reinforcement learning by aligning training and inference routers. *arXiv preprint arXiv:2510.11370*, 2025.
- [39] Nicholas Metropolis, Arianna W Rosenbluth, Marshall N Rosenbluth, Augusta H Teller, and Edward Teller. Equation of state calculations by fast computing machines. *The journal of chemical physics*, 21(6):1087–1092, 1953.
- [40] W Keith Hastings. Monte carlo sampling methods using markov chains and their applications. 1970.
- [41] Thang Luong, Dawsen Hwang, Hoang H. Nguyen, Golnaz Ghiasi, Yuri Chervonyi, Insuk Seo, Junsu Kim, Garrett Bingham, Jonathan Lee, Swaroop Mishra, Alex Zhai, Clara Huiyi Hu, Henryk Michalewski, Jimin Kim, Jeonghyun Ahn, Junhwi Bae, Xingyou Song, Trieu H. Trinh, Quoc V. Le, and Junehyuk Jung. Towards robust mathematical reasoning, 2025.
- [42] Jingcheng Hu, Yinmin Zhang, Shijie Shang, Xiaobo Yang, Yue Peng, Zhewei Huang, Hebin Zhou, Xin Wu, Jie Cheng, Fanqi Wan, Xiangwen Kong, Chengyuan Yao, Kaiwen Yan, Ailin Huang, Hongyu Zhou, Qi Han, Zheng Ge, Daxin Jiang, Xiangyu Zhang, and Heung-Yeung Shum. Pacore: Learning to scale test-time compute with parallel coordinated reasoning, 2026.
- [43] Building effective agents. <https://www.anthropic.com/engineering/building-effective-agents>.
- [44] Unrolling the codex agent loop. <https://openai.com/index/unrolling-the-codex-agent-loop/>.
- [45] Charlie Victor Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling parameters for reasoning. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [46] Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel Candès, and Tatsunori B Hashimoto. s1: Simple test-time scaling. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 20286–20332, 2025.
- [47] Xinyu Yang, Yuwei An, Hongyi Liu, Tianqi Chen, and Beidi Chen. Multiverse: Your language models secretly decide how to parallelize and merge generation. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.
- [48] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv:2004.05150*, 2020.
- [49] Gemma Team, Aishwarya Kamath, Johan Ferret, et al. Gemma 3 technical report, 2025.
- [50] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning, ICML’23. JMLR.org*, 2023.
- [51] Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *International conference on machine learning*, pages 9355–9366. PMLR, 2021.

- [52] Jikai Wang, Yi Su, Juntao Li, Qingrong Xia, Zi Ye, Xinyu Duan, Zhefeng Wang, and Min Zhang. Opt-tree: Speculative decoding with adaptive draft tree structure. *Transactions of the Association for Computational Linguistics*, 13:188–199, 2025.
- [53] Yunfan Xiong, Ruoyu Zhang, Yanzeng Li, and Lei Zou. Dyspec: Faster speculative decoding with dynamic token tree structure. *World Wide Web*, 28(3):36, 2025.
- [54] Haoran You, Yichao Fu, Zheng Wang, Amir Yazdanbakhsh, and Yingyan (Celine) Lin. When linear attention meets autoregressive decoding: towards more effective and efficient linearized large language models. In *Proceedings of the 41st International Conference on Machine Learning, ICML’24*. JMLR.org, 2024.
- [55] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebron, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 4895–4901, Singapore, December 2023. Association for Computational Linguistics.
- [56] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. In *International Conference on Machine Learning*, 2024.
- [57] Gemma Team, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, Shreya Pathak, Laurent Sifre, Morgane Rivière, Mihir Sanjay Kale, Juliette Love, et al. Gemma: Open models based on gemini research and technology, 2024.
- [58] Team Cohere, :, Aakanksha, Arash Ahmadian, Marwan Ahmed, et al. Command a: An enterprise-ready large language model, 2025.
- [59] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *The Twelfth International Conference on Learning Representations*, 2024.
- [60] Mingjie Sun, Xinlei Chen, J Zico Kolter, and Zhuang Liu. Massive activations in large language models. In *First Conference on Language Modeling*, 2024.
- [61] Xiangming Gu, Tianyu Pang, Chao Du, Qian Liu, Fengzhuo Zhang, Cunxiao Du, Ye Wang, and Min Lin. When attention sink emerges in language models: An empirical view. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [62] John Jumper, Richard Evans, Alexander Pritzel, Tim Green, Michael Figurnov, Olaf Ronneberger, Kathryn Tunyasuvunakool, Russ Bates, Augustin Žídek, Anna Potapenko, Alex Bridgland, Clemens Meyer, Simon A. A. Kohl, Andrew J. Ballard, Andrew Cowie, Bernardino Romera-Paredes, Stanislav Nikolov, Rishub Jain, Jonas Adler, Trevor Back, Stig Petersen, David Reiman, Ellen Clancy, Michal Zielinski, Martin Steinegger, Michalina Pacholska, Tamas Berghammer, Sebastian Bodenstein, David Silver, Oriol Vinyals, Andrew W. Senior, Koray Kavukcuoglu, Pushmeet Kohli, and Demis Hassabis. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596(7873):583–589, August 2021.
- [63] Zhixuan Lin, Evgenii Nikishin, Xu He, and Aaron Courville. Forgetting transformer: Softmax attention with a forget gate. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [64] Lean Wang, Huazuo Gao, Chenggang Zhao, Xu Sun, and Damai Dai. Auxiliary-loss-free load balancing strategy for mixture-of-experts. *arXiv preprint arXiv:2408.15664*, 2024.
- [65] Yuxuan Cai, Xiaozhuan Liang, Xinghua Wang, Jin Ma, Haijin Liang, Jinwen Luo, Xinyu Zuo, Lisheng Duan, Yuyang Yin, and Xi Chen. Fastmtp: Accelerating llm inference with enhanced multi-token prediction, 2025.
- [66] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, Geeta Chauhan, Anjali Chourdia, Will Constable, Alban Desmaison, Zachary DeVito, Elias Ellison, Will Feng, Jiong Gong, Michael Gschwind, Brian Hirsh, Sherlock Huang, Kshiteej Kalambarkar, Laurent Kirsch, Michael Lazos, Mario Lezcano, Yanbo Liang, Jason Liang, Yinghai Lu, CK Luk, Bert Maher, Yunjie Pan,

- Christian Puhersch, Matthias Reso, Mark Saroufim, Marcos Yukio Siraichi, Helen Suk, Michael Suo, Phil Tillet, Eikan Wang, Xiaodong Wang, William Wen, Shunting Zhang, Xu Zhao, Keren Zhou, Richard Zou, Ajit Mathews, Gregory Chanan, Peng Wu, and Soumith Chintala. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. In *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)*. ACM, April 2024.
- [67] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- [68] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Anand Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. Efficient large-scale language model training on gpu clusters using megatron-lm, 2021.
- [69] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. Zero: Memory optimizations toward training trillion parameter models. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–16, 2020.
- [70] Dennis Liu, Zijie Yan, Xin Yao, Tong Liu, Vijay Korthikanti, Evan Wu, Shiqing Fan, Gao Deng, Hongxiao Bai, Jianbin Chang, Ashwath Aithal, Michael Andersch, Mohammad Shoeybi, Jiajie Yao, Chandler Zhou, David Wu, Xipeng Li, and June Yang. Moe parallel folding: Heterogeneous parallelism mappings for efficient large-scale moe model training with megatron core, 2025.
- [71] Wentao Guo, Mayank Mishra, Xinle Cheng, Ion Stoica, and Tri Dao. Sonicmoe: Accelerating moe with io and tile-aware optimizations, 2025.
- [72] Jeremy Bernstein and Laker Newhouse. Old optimizer, new norm: An anthology, 2024.
- [73] Noah Amsel, David Persson, Christopher Musco, and Robert M. Gower. The polar express: Optimal matrix sign methods and their application to the muon algorithm, 2025.
- [74] Zihan Qiu, Zeyu Huang, Bo Zheng, Kaiyue Wen, Zekun Wang, Rui Men, Ivan Titov, Dayiheng Liu, Jingren Zhou, and Junyang Lin. Demons in the detail: On implementing load balancing loss for training specialized mixture-of-expert models. *arXiv preprint arXiv:2501.11873*, 2025.
- [75] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruizhe Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [76] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. 2019.
- [77] Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tieyan Liu. On layer normalization in the transformer architecture. In *International conference on machine learning*, pages 10524–10533. PMLR, 2020.
- [78] Noam Shazeer. Glue variants improve transformer, 2020.
- [79] Common Crawl. Common crawl. <https://commoncrawl.org>.
- [80] Siming Huang, Tianhao Cheng, J. K. Liu, Jiaran Hao, Liuyihan Song, Yang Xu, J. Yang, Jiaheng Liu, Chenchen Zhang, Linzheng Chai, Ruifeng Yuan, Zhaoxiang Zhang, Jie Fu, Qian Liu, Ge Zhang, Zili Wang, Yuan Qi, Yinghui Xu, and Wei Chu. Opencoder: The open cookbook for top-tier code large language models, 2025.

- [81] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024.
- [82] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying llm-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.
- [83] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding, 2024.
- [84] Wenhan Xiong, Jingyu Liu, Igor Molybog, Hejia Zhang, Prajjwal Bhargava, Rui Hou, Louis Martin, Rashi Rungta, Karthik Abinav Sankararaman, Barlas Oguz, et al. Effective long-context scaling of foundation models, 2024.
- [85] Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, et al. Deepseek-v3. 2: Pushing the frontier of open large language models. *arXiv preprint arXiv:2512.02556*, 2025.
- [86] Yixin Ye, Zhen Huang, Yang Xiao, Ethan Chern, Shijie Xia, and Pengfei Liu. Limo: Less is more for reasoning, 2025.
- [87] Fabio Pardo, Arash Tavakoli, Vitaly Levnik, and Petar Kormushev. Time limits in reinforcement learning, 2022.
- [88] Michael Luo, Sijun Tan, Justin Wong, Xiaoxiang Shi, William Y. Tang, Manan Roongta, Colin Cai, Jeffrey Luo, Li Erran Li, Raluca Ada Popa, and Ion Stoica. Deepscaler: Surpassing o1-preview with a 1.5b model by scaling rl. <https://pretty-radio-b75.notion.site/DeepScaler-Surpassing-O1-Preview-with-a-1-5B-Model-by-Scaling-RL-19681902c1468005bed8ca303013a4e2>, 2025. Notion Blog.
- [89] Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.
- [90] Jingcheng Hu, Yinmin Zhang, Qi Han, Daxin Jiang, Xiangyu Zhang, and Heung-Yeung Shum. Open-Reasoner-Zero: An open source approach to scaling up reinforcement learning on the base model. *arXiv preprint arXiv:2503.24290*, 2025.
- [91] Minh-Thang Luong, Dawsen Hwang, Hoang H Nguyen, Golnaz Ghiasi, Yuri Chervonyi, Insuk Seo, Junsu Kim, Garrett Bingham, Jonathan Lee, Swaroop Mishra, et al. Towards robust mathematical reasoning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 35406–35430, 2025.
- [92] François Chollet. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019.
- [93] Long Phan, Alice Gatti, Ziwen Han, Nathaniel Li, Josephina Hu, Hugh Zhang, Chen Bo Calvin Zhang, Mohamed Shaaban, John Ling, Sean Shi, et al. Humanity’s last exam, 2025.
- [94] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024.
- [95] Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback, 2022.
- [96] Lunjun Zhang, Arian Hosseini, Hritik Bansal, Mehran Kazemi, Aviral Kumar, and Rishabh Agarwal. Generative verifiers: Reward modeling as next-token prediction, 2025.
- [97] Ralph Allan Bradley and Milton E Terry. Rank analysis of incomplete block designs: I. the method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952.



- [98] Chen Hu, Haikuo Du, Heng Wang, Lin Lin, Mingrui Chen, Peng Liu, Ruihang Miao, Tianchi Yue, Wang You, Wei Ji, Wei Yuan, Wenjin Deng, Xiaojian Yuan, Xiaoyun Zhang, Xiangyu Liu, Xikai Liu, Yanming Xu, Yicheng Cao, Yifei Zhang, Yongyao Wang, Yubo Shu, Yurong Zhang, Yuxiang Zhang, Zheng Gong, Zhichao Chang, Binyan Li, Dan Ma, Furong Jia, Hongyuan Wang, Jiayu Liu, Jing Bai, Junlan Liu, Manjiao Liu, Na Wang, Qiuping Wu, Qinxin Du, Shiwei Li, Wen Sun, Yifeng Gong, Yonglin Chen, Yuling Zhao, Yuxuan Lin, Ziqi Ren, Zixuan Wang, Aihu Zhang, Brian Li, Buyun Ma, Kang An, Li Xie, Mingliang Li, Pan Li, Shidong Yang, Xi Chen, Xiaojia Liu, Yuchu Luo, Yuan Song, Yuanhao Ding, Yuanwei Liang, Zexi Li, Zhaoning Zhang, Zixin Zhang, Binxing Jiao, Daxin Jiang, Jiansheng Chen, Jing Li, Xiangyu Zhang, and Yibo Zhu. Step-deepresearch technical report, 2025.
- [99] Jia Li, Edward Beeching, Lewis Tunstall, Ben Lipkin, Roman Soletskyi, Shengyi Huang, Kashif Rasul, Longhui Yu, Albert Q Jiang, Ziju Shen, et al. Numinamath: The largest public dataset in ai4maths with 860k pairs of competition math problems and solutions. 2024.
- [100] Alon Albalak et al. Big-math: A large-scale, high-quality math dataset for reinforcement learning in language models. *arXiv preprint arXiv:2502.17387*, 2025.
- [101] Arindam Mitra, Hamed Khanpour, Corby Rosset, and Ahmed Awadallah. Orca-math: Unlocking the potential of slms in grade school math. *arXiv preprint arXiv:2402.14830*, 2024.
- [102] aslawliet. Olympiads. <https://huggingface.co/datasets/aslawliet/olympiads>, 2024. Hugging Face dataset.
- [103] aslawliet. Cn-k12. <https://huggingface.co/datasets/aslawliet/cn-k12>, 2024. Hugging Face dataset of Chinese K-12 math problems.
- [104] Open-R1 Team. Openr1-math-220k. <https://huggingface.co/datasets/open-r1/OpenR1-Math-220k>, 2025. Open-source distilled math reasoning dataset.
- [105] X. He et al. Deepmath-103k: A large-scale, challenging math qa benchmark. *arXiv preprint arXiv:2504.11456*, 2025.
- [106] Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Weinan Dai, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Mu Qiao, Yonghui Wu, and Mingxuan Wang. DAPO: An open-source LLM reinforcement learning system at scale, 2025.
- [107] Etash Guha et al. Openthoughts: Data recipes for reasoning models. *arXiv preprint arXiv:2506.04178*, 2025.
- [108] Niklas Muennighoff et al. s1: Simple test-time scaling. <https://arxiv.org/abs/2501.19393>, 2025.
- [109] Yunjie Ji, Xiaoyu Tian, Sitong Zhao, Haotian Wang, Shuaiting Chen, Yiping Peng, Han Zhao, and Xiangang Li. Am-thinking-v1: Advancing the frontier of reasoning at 32b scale, 2025.
- [110] LIMO Authors. Less is more for reasoning: Semi-parametric math reasoners. *arXiv preprint arXiv:2502.03387*, 2025.
- [111] Rongao Li, Jie Fu, Bo-Wen Zhang, Tao Huang, Zhihong Sun, Chen Lyu, Guang Liu, Zhi Jin, and Ge Li. Taco: Topics in algorithmic code generation dataset, 2023.
- [112] Michael Luo, Sijun Tan, Roy Huang, Ameen Patel, Alpay Ariyak, Qingyang Wu, Xiaoxiang Shi, Rachel Xin, Colin Cai, Maurice Weber, Ce Zhang, Li Erran Li, Raluca Ada Popa, and Ion Stoica. Deepcoder: A fully open-source 14b coder at o3-mini level. <https://www.together.ai/blog/deepcoder>, 2025. Technical Blog.
- [113] Zihan Wang, Siyao Liu, Yang Sun, Hongyan Li, and Kai Shen. Codecontests+: High-quality test case generation for competitive programming, 2025.

- [114] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. Camel: Communicative agents for "mind" exploration of large scale language model society, 2023.
- [115] Akhiad Bercovich, Itay Levy, Izik Golan, Mohammad Dabbah, Ran El-Yaniv, Omri Puny, Ido Galil, Zach Moshe, Tomer Ronen, Najeeb Nabwani, et al. Llama-nemotron: Efficient reasoning models, 2025.
- [116] Run-Ze Fan, Zengzhi Wang, and Pengfei Liu. Megascience: Pushing the frontiers of post-training datasets for science reasoning. *arXiv preprint arXiv:2507.16812*, 2025.
- [117] Wenting Zhao, Xiang Ren, Jack Hessel, Claire Cardie, Yejin Choi, and Yuntian Deng. Wildchat: 1m chatgpt interaction logs in the wild. *arXiv preprint arXiv:2405.01470*, 2024.
- [118] Junteng Liu, Yunji Li, Chi Zhang, Jingyang Li, Aili Chen, Ke Ji, Weiyu Cheng, Zijia Wu, Chengyu Du, Qidi Xu, et al. Webexplorer: Explore and evolve for training long-horizon web agents. *arXiv preprint arXiv:2509.06501*, 2025.
- [119] Kuan Li, Zhongwang Zhang, Huifeng Yin, Liwen Zhang, Litu Ou, Jialong Wu, Wenbiao Yin, Baixuan Li, Zhengwei Tao, Xinyu Wang, et al. Websailor: Navigating super-human reasoning for web agent. *arXiv preprint arXiv:2507.02592*, 2025.
- [120] Yuetai Li, Huseyin A Inan, Xiang Yue, Wei-Ning Chen, Lukas Wutschitz, Janardhan Kulkarni, Radha Poovendran, Robert Sim, and Saravan Rajmohan. Simulating environments with reasoning models for agent training. *arXiv preprint arXiv:2511.01824*, 2025.
- [121] Lianghong Guo, Yanlin Wang, Caihua Li, Wei Tao, Pengyu Yang, Jiachi Chen, Haoyu Song, Duyu Tang, and Zibin Zheng. Swe-factory: Your automated factory for issue resolution training data and evaluation benchmarks, 2026.
- [122] Jiaran Zhang, Luck Ma, Yanhao Li, Fanqi Wan, Di Qi, Xu Zhao, Jieyi Hou, Zhe Xie, Mengqiang Ren, Xin Wu, Zhewei Huang, Liangyu Chen, Yingwei Ma, Qi Han, and Xiangyu Zhang. Docksmith: Scaling reliable coding environments via an agentic docker builder, 2026.
- [123] John Yang, Kilian Lieret, Carlos E Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. Swe-smith: Scaling data for software engineering agents. *arXiv preprint arXiv:2504.21798*, 2025.
- [124] Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. Training software engineering agents and verifiers with swe-gym. *arXiv preprint arXiv:2412.21139*, 2024.
- [125] Naman Jain, Jaskirat Singh, Manish Shetty, Liang Zheng, Koushik Sen, and Ion Stoica. R2e-gym: Procedural environments and hybrid verifiers for scaling open-weights swe agents. *arXiv preprint arXiv:2504.07164*, 2025.
- [126] Ibragim Badertdinov, Alexander Golubev, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Andrei Andriushchenko, Maria Trofimova, Daria Litvintseva, and Boris Yangel. Swe-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents. *arXiv preprint arXiv:2505.20411*, 2025.
- [127] Qijia Shen, Jay Rainton, Aznaur Aliev, Ahmed Awelkair, Boyuan Ma, Zhiqi Huang, Yuzhen Mao, Wendong Fan, Philip Torr, Bernard Ghanem, Changran Hu, Urmish Thakker, and Guohao Li. SETA: Scaling Environments for Terminal Agents, January 2026.
- [128] Xiaozhi Wang, Tianyu Gao, Zhaocheng Zhu, Zhengyan Zhang, Zhiyuan Liu, Juanzi Li, and Jian Tang. Kepler: A unified model for knowledge embedding and pre-trained language representation. *Transactions of the Association for Computational Linguistics*, 9:176–194, 2021.
- [129] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638, September 2025.

- [130] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, 2025.
- [131] Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. Openhands: An open platform for ai software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.
- [132] John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652, 2024.
- [133] Inc. Kilo Code. Move at kilo speed. <https://kilo.ai/>, 2026. Kilo Code webpage.
- [134] Roo Code. Your ai software engineering team is here. <https://roocode.com/>, 2026. Roo Code webpage.
- [135] ANTHROPIC PBC. Autocomplete finishes lines. claude code finishes features. <https://claude.com/product/claude-code>, 2026. Claude Code webpage.
- [136] Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V. Le, Ed H. Chi, Denny Zhou, and Jason Wei. Challenging big-bench tasks and whether chain-of-thought can solve them, 2022.
- [137] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding, 2020.
- [138] Aryo Pradipta Gema, Joshua Ong Jun Leang, Giwon Hong, Alessio Devoto, Alberto Carlo Maria Mancino, Rohit Saxena, Xuanli He, Yu Zhao, Xiaotang Du, Mohammad Reza Ghasemi Madani, Claire Barale, Robert McHardy, Joshua Harris, Jean Kaddour, Emile van Krieken, and Pasquale Minervini. Are we done with mmlu?, 2024.
- [139] Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyang Jiang, Tianle Li, Max Ku, Kai Wang, Alex Zhuang, Rongqi Fan, Xiang Yue, and Wenhui Chen. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark, 2024.
- [140] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence?, 2019.
- [141] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: An adversarial winograd schema challenge at scale, 2019.
- [142] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. Gpqa: A graduate-level google-proof q&a benchmark, 2023.
- [143] Xinrun Du, Yifan Yao, Kaijing Ma, Bingli Wang, Tianyu Zheng, King Zhu, Minghao Liu, Yiming Liang, Xiaolong Jin, Zhenlin Wei, et al. Supergpqa: Scaling llm evaluation across 285 graduate disciplines, 2025.
- [144] OpenAI. Simpleqa. <https://github.com/openai/simple-evals>, 2024.
- [145] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021.

- [146] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset, 2021.
- [147] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021.
- [148] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models, 2021.
- [149] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation, 2023.
- [150] Federico Cassano, John Gouwar, Daniel Nguyen, Sydney Nguyen, Luna Phipps-Costin, Donald Pinckney, Ming-Ho Yee, Yangtian Zi, Carolyn Jane Anderson, Molly Q Feldman, Arjun Guha, Michael Greenberg, and Abhinav Jangda. Multipl-e: A scalable and extensible approach to benchmarking neural code generation, 2022.
- [151] Yuzhen Huang, Yuzhuo Bai, Zhihao Zhu, Junlei Zhang, Jinghan Zhang, Tangjun Su, Junteng Liu, Chuancheng Lv, Yikai Zhang, Jiayi Lei, Yao Fu, Maosong Sun, and Junxian He. C-eval: A multi-level multi-discipline chinese evaluation suite for foundation models, 2023.
- [152] Haonan Li, Yixuan Zhang, Fajri Koto, Yifei Yang, Hai Zhao, Yeyun Gong, Nan Duan, and Timothy Baldwin. Cmmlu: Measuring massive multitask language understanding in chinese, 2023.
- [153] Yancheng He, Shilong Li, Jiaheng Liu, Yingshui Tan, Weixun Wang, Hui Huang, Xingyuan Bu, Hangyu Guo, Chengwei Hu, Boren Zheng, et al. Chinese simpleqa: A chinese factuality evaluation for large language models, 2024.
- [154] Long Phan, Tony CY Pang, Adam Wecker, Yifan Xiong, Dan Hendrycks, et al. Humanity’s last exam, 2025.
- [155] Tianle Li, Wei-Lin Chiang, Evan Frick, Lisa Dunlap, Banghua Zhu, Joseph E. Gonzalez, and Ion Stoica. From live data to high-quality benchmarks: The arena-hard pipeline, April 2024.
- [156] Valentina Pyatkin, Saumya Malik, Victoria Graf, Hamish Ivison, Shengyi Huang, Pradeep Dasigi, Nathan Lambert, and Hannaneh Hajishirzi. Generalizing verifiable instruction following. *arXiv preprint arXiv:2507.02833*, 2025.
- [157] Ved Sirdeshmukh, Kaustubh Deshpande, Johannes Mols, Lifeng Jin, Ed-Yeremai Cardona, Dean Lee, Jeremy Kritz, Willow Primack, Summer Yue, and Chen Xing. Multichallenge: A realistic multi-turn conversation evaluation benchmark challenging to frontier llms, 2025.
- [158] Yushi Bai, Shangqing Tu, Jiajie Zhang, Hao Peng, Xiaozhi Wang, Xin Lv, Shulin Cao, Jiazheng Xu, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. Longbench v2: Towards deeper understanding and reasoning on realistic long-context multitasks, 2024.
- [159] Kiran Vodrahalli, Santiago Ontanon, Nilesch Tripuraneni, Kelvin Xu, Sanil Jain, Rakesh Shrivastava, Jeffrey Hui, Nishanth Dikkala, Mehran Kazemi, Bahare Fatemi, et al. Michelangelo: Long context evaluations beyond haystacks via latent structure queries. *arXiv preprint arXiv:2409.12640*, 2024.

- [160] Satyapriya Krishna, Kalpesh Krishna, Anhad Mohananey, Steven Schwarcz, Adam Stambler, Shyam Upadhyay, and Manaal Faruqui. Fact, fetch, and reason: A unified evaluation of retrieval-augmented generation. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 4745–4759, 2025.
- [161] Jiawei Liu, Jia Le Tian, Vijay Daita, Yuxiang Wei, Yifeng Ding, Yuhan Katherine Wang, Jun Yang, and Lingming Zhang. Repoqa: Evaluating long context code understanding, 2024.
- [162] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. Yarn: Efficient context window extension of large language models, 2023.
- [163] Tianyu Gao, Alexander Wettig, Luxi He, Yihe Dong, Sadhika Malladi, and Danqi Chen. Metadata conditioning accelerates language model pre-training. In *International Conference on Machine Learning (ICML)*, 2025.
- [164] Zeyuan Allen-Zhu and Yuanzhi Li. Physics of language models: Part 3.3, knowledge capacity scaling laws. *arXiv preprint arXiv:2404.05405*, 2024.
- [165] Dongyang Fan, Diba Hashemi, Sai Praneeth Karimireddy, and Martin Jaggi. Beyond urls: Metadata diversity and position for efficient llm pretraining, 2025.
- [166] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [167] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- [168] Colin White, Samuel Dooley, Manley Roberts, Arka Pal, Ben Feuer, Siddhartha Jain, Ravid Shwartz-Ziv, Neel Jain, Khalid Saifullah, Sreemanti Dey, Shubh-Agrawal, Sandeep Singh Sandha, Siddhartha Naidu, Chinmay Hegde, Yann LeCun, Tom Goldstein, Willie Neiswanger, and Micah Goldblum. Livebench: A challenging, contamination-limited llm benchmark, 2025.
- [169] MAA. American invitational mathematics examination - aime. In *American Invitational Mathematics Examination - AIME*, 2024.
- [170] MAA. American invitational mathematics examination - aime. In *American Invitational Mathematics Examination - AIME*, 2025.
- [171] Mislav Balunović, Jasper Dekoninck, Ivo Petrov, Nikola Jovanović, and Martin Vechev. Matharena: Evaluating llms on uncontaminated math competitions. *arXiv preprint arXiv:2505.23281*, 2025.
- [172] Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. Instruction-following evaluation for large language models, 2023.
- [173] Bill Yuchen Lin, Yuntian Deng, Khyathi Chandu, Faeze Brahman, Abhilasha Ravichander, Valentina Pyatkin, Nouha Dziri, Ronan Le Bras, and Yejin Choi. Wildbench: Benchmarking llms with challenging tasks from real users in the wild, 2024.
- [174] Cheng-Ping Hsieh, Simeng Sun, Samuel Krman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. Ruler: What’s the real context size of your long-context language models?, 2024.
- [175] Howard Yen, Tianyu Gao, Minmin Hou, Ke Ding, Daniel Fleischer, Peter Izsak, Moshe Wasserblat, and Danqi Chen. Helmet: How to evaluate long-context language models effectively and thoroughly, 2024.
- [176] Yang Zhou, Hongyi Liu, Zhuoming Chen, Yuandong Tian, and Beidi Chen. Gsm-infinite: How do your llms behave over infinitely increasing context length and reasoning complexity?, 2025.

- [177] Yongqi An, Xu Zhao, Tao Yu, Ming Tang, and Jinqiao Wang. Systematic outliers in large language models. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [178] Alexander Wettig, Kyle Lo, Sewon Min, Hannaneh Hajishirzi, Danqi Chen, and Luca Soldaini. Organize the web: Constructing domains enhances pre-training data curation, 2025.
- [179] Dan Su, Kezhi Kong, Ying Lin, Joseph Jennings, Brandon Norick, Markus Kliegl, Mostofa Patwary, Mohammad Shoeybi, and Bryan Catanzaro. Nemotron-cc: Transforming common crawl into a refined long-horizon pretraining dataset, 2025.
- [180] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024.
- [181] Fan Zhou, Zengzhi Wang, Nikhil Ranjan, Zhoujun Cheng, Liping Tang, Guowei He, Zhengzhong Liu, and Eric P. Xing. Megamath: Pushing the limits of open math corpora, 2025.
- [182] Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Gabriel Martín Blázquez, Guilherme Penedo, Lewis Tunstall, Andrés Marafioti, Hynek Kydlíček, Agustín Piqueres Lajarín, Vaibhav Srivastav, Joshua Lochner, Caleb Fahlgren, Xuan-Son Nguyen, Clémentine Fourrier, Ben Burtenshaw, Hugo Larcher, Haojun Zhao, Cyril Zakka, Mathieu Morlon, Colin Raffel, Leandro von Werra, and Thomas Wolf. Smollm2: When smol goes big – data-centric training of a small language model, 2025.
- [183] Zihan Zheng, Zerui Cheng, Zeyu Shen, Shang Zhou, Kaiyuan Liu, Hansen He, Dongruixuan Li, Stanley Wei, Hangyi Hao, Jianzhu Yao, et al. Livecodebench pro: How do olympiad medalists judge llms in competitive programming? *arXiv preprint arXiv:2506.11928*, 2025.
- [184] Anonymous. Autocode: LLMs as problem setters for competitive programming. In *The Fourteenth International Conference on Learning Representations*, 2026.
- [185] Shanghaoran Quan, Jiayi Yang, Bowen Yu, Bo Zheng, Dayiheng Liu, An Yang, Xuancheng Ren, Bofei Gao, Yibo Miao, Yunlong Feng, Zekun Wang, Jian Yang, Zeyu Cui, Yang Fan, Yichang Zhang, Binyuan Hui, and Junyang Lin. Codeelo: Benchmarking competition-level code generation of llms with human-comparable elo ratings, 2025.
- [186] OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report, 2024.
- [187] OpenAI. Openai o3-mini, 2025.
- [188] OpenAI. Introducing gpt-5, 2025.
- [189] Haolong Yan, Jia Wang, Xin Huang, Yeqing Shen, Ziyang Meng, Zhimin Fan, Kaijun Tan, Jin Gao, Lieyu Shi, Mi Yang, et al. Step-gui technical report. *arXiv preprint arXiv:2512.15431*, 2025.